

Module Technologies java et Neo4j

Année universitaire : 2025/2026

Rapport TP1 : POO JAVA

Effectuée par :

EL KYSY Aya

Professeur :

AMAMOU Ahmed

Travaux pratiques : série d'exercices

EXO N 0 : ***Gestion d'une médiathèque***

1. Les noms et les verbes qui deviennent des classes et des méthodes

Les noms qui deviennent des classes :

Médiathèque

Document

Livre

DVD

Adhérent

Adresse

Les verbes qui deviennent des méthodes :

estDisponible() : vérifier si un document est disponible.

emprunter() : emprunter un document.

rendre() : rendre un document.

afficherEmprunts() : afficher la liste des emprunts d'un adhérent.

Classe	Attributs	Type Java
Document	cote, titre, anneeParution, disponible	String, String, int, boolean
Livre	auteur, nombrePages	String, int
DVD	realisateur, duree	String, int
Adherent	numero, nom, adresse, emprunts	int, String, Adresse, Document[]
Adresse	rue, ville, codePostal	String, String, String
Mediatheque	documents, adherents	Document[], Adherent[]

Classe 1	Classe 2	Relation	Justification
Livre	Document	Est un	Un livre est un type de document.
DVD	Document	Est un	Un DVD est un type de document.
Adherent	Adresse	A un	Un adhérent possède une adresse.
Adherent	Document	Emprunte	Un adhérent peut emprunter jusqu'à 3 documents.
Mediatheque	Document	Contient	La médiathèque contient plusieurs documents.
Mediatheque	Adherent	Contient	La médiathèque gère plusieurs adhérents.
Livre	DVD	Aucune	Ce sont deux types différents de documents.

4. Les attributs et méthodes de la classe parent commune

La classe parent commune est Document, car Livre et DVD sont deux types de documents.

Attributs communs :

cote

titre

anneeParution

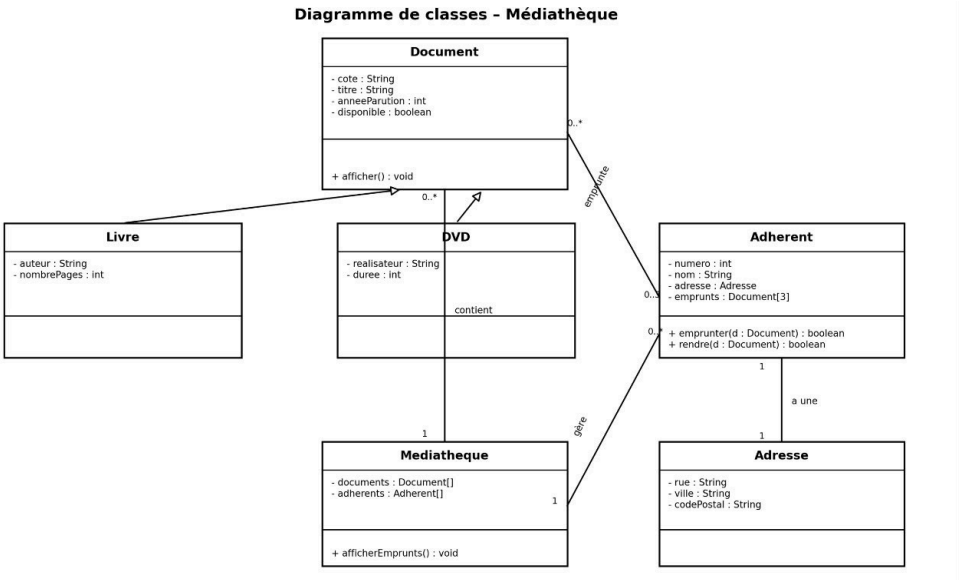
disponible

Méthode commune :

- estDisponible()

On crée une classe parent Document afin de regrouper les attributs et les méthodes communs aux classes Livre et DVD. Cela permet d'éviter la répétition du code et de faciliter la maintenance du programme. Chaque classe enfant possède ensuite ses propres attributs spécifiques.

5 -



6- Le nombre maximum d'emprunts peut être défini comme une constante MAX_EMPRUNTS = 3 dans la classe Adherent, car il représente une règle générale de la médiathèque.

Cependant, si les adhérents premium peuvent emprunter jusqu'à 6 documents, il est préférable de rendre cette limite configurable ou de créer une classe AdherentPremium qui hérite de Adherent et possède une limite différente.

Exo n 1: La classe Livre

```java
public class Livre {  6 usages

    // Declaration des attributs
    private String titre;  2 usages
    private String auteur;  2 usages
    private int nombrePages;  3 usages
    private double prix;  6 usages

    // Constructeur avec les quatre parametres
    public Livre(String titre, String auteur, int nombrePages, double prix) {  3 usages
        this.titre = titre;
        this.auteur = auteur;
        this.nombrePages = nombrePages;
        this.prix = prix;
    }

    // Methode pour afficher les informations du livre
    public void afficher() {  6 usages
        System.out.println("Titre : " + titre);
        System.out.println("Auteur : " + auteur);
        System.out.println("Nombre de pages : " + nombrePages);
        System.out.println("Prix : " + prix + " DH");
    }

    // Methode qui verifie si le livre depasse 300 pages
    public boolean estLong() {  3 usages
        if (nombrePages > 300) {
            return true;
        } else {
            return false;
        }
    }
```

Microsoft Defender may affect
To avoid performance issues, exclu
and the project folders from the Re
Exclude folders
More

```java
// Methode qui verifie si le livre depasse 300 pages
public boolean estLong() {  3 usages
    if (nombrePages > 300) {
        return true;
    } else {
        return false;
    }
}
// Methode pour appliquer une remise
public void appliquerRemise(double pourcentage) {  2 usages

    if (pourcentage >= 0 && pourcentage <= 50) {

        double remise = prix * pourcentage / 100;
        prix = prix - remise;

        System.out.println("Nouveau prix : " + prix + " DH");

    } else {
        System.out.println("Remise refusee !");
    }
}
}
```

```java
public class Main {
    public static void main(String[] args) {
        Livre livre2 = new Livre( titre: "Les Miserables", auteur: "Victor Hugo", nombrePages: 1900,
        Livre livre3 = new Livre( titre: "L'Etranger", auteur: "Albert Camus", nombrePages: 120, prix:

        // Afficher les trois livres
        System.out.println("=== LES LIVRES ===");

        livre1.afficher();
        System.out.println("Livre long : " + livre1.estLong());

        livre2.afficher();
        System.out.println("Livre long : " + livre2.estLong());

        livre3.afficher();
        System.out.println("Livre long : " + livre3.estL

        // Appliquer une remise de 10% sur le premier li
        livre1.appliquerRemise( pourcentage: 10);

        // Appliquer une remise de 80% sur le deuxieme livre
        livre2.appliquerRemise( pourcentage: 80);

        // Afficher les livres apres les remises
        System.out.println("\n=== APRES LES REMISES ===");

        livre1.afficher();
        livre2.afficher();
        livre3.afficher();
    }
}
```

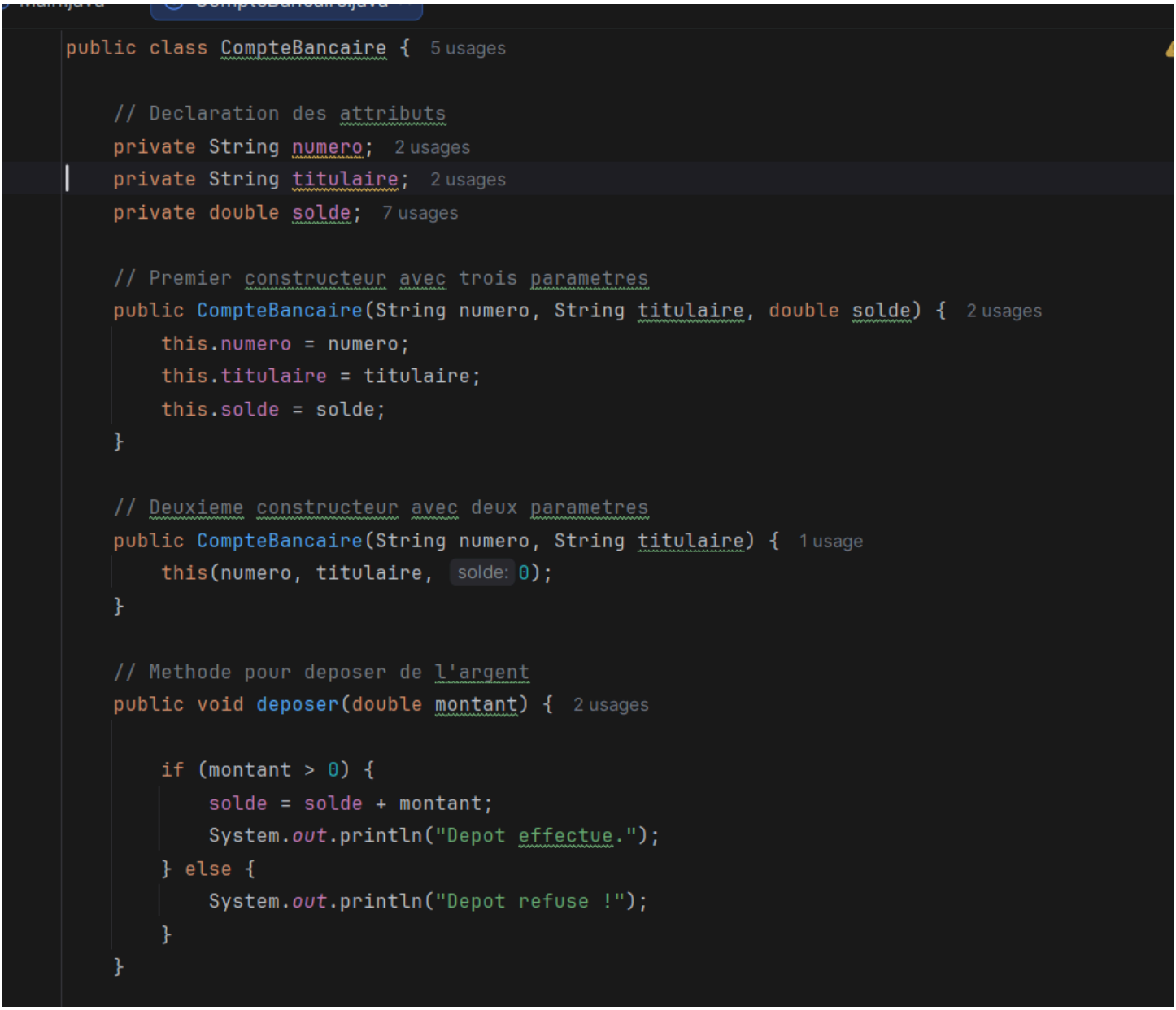
3

Livre

public boolean estLong()

exo1

Exercice 2 – Le compte bancaire



```java
public class CompteBancaire {  5 usages

    // Declaration des attributs
    private String numero;  2 usages
    private String titulaire;  2 usages
    private double solde;  7 usages

    // Premier constructeur avec trois parametres
    public CompteBancaire(String numero, String titulaire, double solde) {  2 usages
        this.numero = numero;
        this.titulaire = titulaire;
        this.solde = solde;
    }

    // Deuxieme constructeur avec deux parametres
    public CompteBancaire(String numero, String titulaire) {  1 usage
        this(numero, titulaire,  solde: 0);
    }

    // Methode pour deposer de l'argent
    public void deposer(double montant) {  2 usages

        if (montant > 0) {
            solde = solde + montant;
            System.out.println("Depot effectue.");
        } else {
            System.out.println("Depot refuse !");
        }
    }
```

```java
public class CompteBancaire {  5 usages

    // Methode pour retirer de l'argent
    public boolean retirer(double montant) {  3 usages

        if (montant > 0 && solde >= montant) {
            solde = solde - montant;
            System.out.println("Retrait effectue.");
            return true;
        } else {
            System.out.println("Retrait refuse !");
            return false;
        }
    }

    // Methode pour transferer de l'argent
    public void virerVers(CompteBancaire destinataire, double montant) {  2 usages

        if (retirer(montant)) {
            destinataire.deposer(montant);
            System.out.println("Virement effectue.");
        } else {
            System.out.println("Virement refuse !");
        }
    }

    // Methode pour afficher les informations du compte
    public void afficher() {  2 usages

        System.out.println("Numero : " + numero);
        System.out.println("Titulaire : " + titulaire);
        System.out.println("Solde : " + solde + " DH");
```

```java
public class CompteBancaire {  5 usages
    public boolean retirer(double montant) {  3 usages
            System.out.println("Retrait refuse !");
            return false;
        }
    }

    // Methode pour transferer de l'argent
    public void virerVers(CompteBancaire destinataire, double montant) {  2 usages

        if (retirer(montant)) {
            destinataire.deposer(montant);
            System.out.println("Virement effectue.");
        } else {
            System.out.println("Virement refuse !");
        }
    }

    // Methode pour afficher les informations du compte
    public void afficher() {  2 usages

        System.out.println("Numero : " + numero);
        System.out.println("Titulaire : " + titulaire);
        System.out.println("Solde : " + solde + " DH");
    }
}
```

Main.java | CompteBancaire.java

```java
public class Main {

    public static void main(String[] args) {

        // Creation des deux comptes
        CompteBancaire A = new CompteBancaire( numero: "MA-001",  titulaire: "Karim",  solde: 1000);
        CompteBancaire B = new CompteBancaire( numero: "MA-002",  titulaire: "Nadia");

        // 1. A retire 300 DH
        System.out.println("1. A retire 300 DH");
        A.retirer( montant: 300);

        // 2. B depose -50 DH
        System.out.println("\n2. B depose -50 DH");
        B.deposer( montant: -50);

        // 3. A vire 500 DH vers B
        System.out.println("\n3. A vire 500 DH vers B");
        A.virerVers(B,  montant: 500);

        // 4. B retire 1000 DH
        System.out.println("\n4. B retire 1000 DH");
        B.retirer( montant: 1000);

        // 5. B vire 100 DH vers A
        System.out.println("\n5. B vire 100 DH vers A");
        B.virerVers(A,  montant: 100);

        // Affichage final des deux comptes
        System.out.println("\n=== SOLDES FINAUX ===");

        A.afficher();
```

```java
        // 4. B retire 1000 DH
        System.out.println("\n4. B retire 1000 DH");
        B.retirer( montant: 1000);

        // 5. B vire 100 DH vers A
        System.out.println("\n5. B vire 100 DH vers A");
        B.virerVers(A,  montant: 100);

        // Affichage final des deux comptes
        System.out.println("\n=== SOLDES FINAUX ===");

        A.afficher();

        System.out.println();

        B.afficher();
    }
}
```

Question de réflexion:

En Java, lorsqu'on passe un objet en paramètre, on transmet une copie de sa référence. Cette référence désigne le même objet en mémoire.

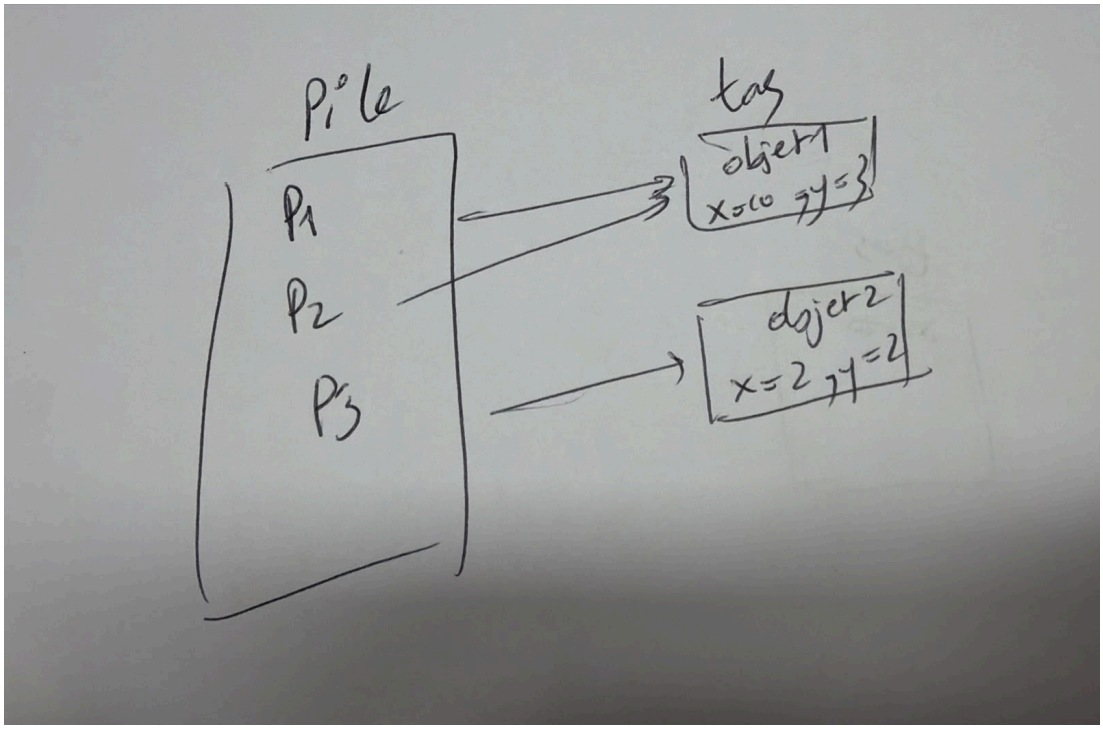
Ainsi, destinataire et B désignent le même compte bancaire. Donc, lorsque nous appelons destinataire.deposer(montant), le solde du compte B est directement modifié.

EXO N 3 : Prédire avant d'exécuter : les références

1-les lignes

A- 10 ,B-true,c- false,D-3,4,E-10

2-



3-Explication des erreurs de raisonnement possibles :

Ligne A : On pourrait penser que p1.x vaut 2, car c'est la valeur initiale. Mais c'est faux, car p1 et p2 désignent le même objet. La modification p2.x = 10 change donc aussi p1.x. Le résultat est 10.

Ligne B : On pourrait penser que p1 == p2 retourne false, car ce sont deux variables différentes. Mais c'est faux, car elles possèdent la même référence et désignent le même objet. Le résultat est true.

Ligne C : On pourrait penser que p1 == p3 retourne true, car les deux objets ont été créés avec les mêmes coordonnées. Mais c'est faux, car ils sont deux objets différents en mémoire. Le résultat est false.

Ligne D : On pourrait penser que les coordonnées restent (2, 3). Mais c'est faux, car la méthode deplacer() ajoute 1 à chaque coordonnée. Le résultat est 3, 4.

Ligne E : On pourrait penser que p1.x devient 0 après l'appel de reinitialiser(p1). Mais c'est faux, car cette méthode modifie uniquement la référence locale p et ne change pas l'objet original. Le résultat reste 10.

Conclusion :

Les erreurs de raisonnement proviennent principalement de la confusion entre les références et les objets. En Java, deux variables peuvent désigner le même objet, et modifier une référence locale ne signifie pas modifier la référence originale.

4-La méthode deplacer() modifie directement les attributs de l'objet reçu en paramètre.

La variable locale p possède une copie de la référence de p3. Elle désigne donc le même objet que p3.

Par conséquent, les modifications effectuées sur p.x et p.y sont visibles dans p3.

Deuxième méthode : reinitialiser()

La méthode reinitialiser() crée un nouvel objet avec les coordonnées (0, 0), puis affecte sa référence à la variable locale p.

Cette opération ne modifie pas la référence originale de p1.

Ainsi, après l'exécution de la méthode, p1 pointe toujours vers son ancien objet avec les coordonnées (10, 3).

Explication de la différence :

deplacer() modifie directement l'objet existant.

reinitialiser() change uniquement la référence locale p.

5- *Objet 1 new Point(2, 3) pour p1 Accessible

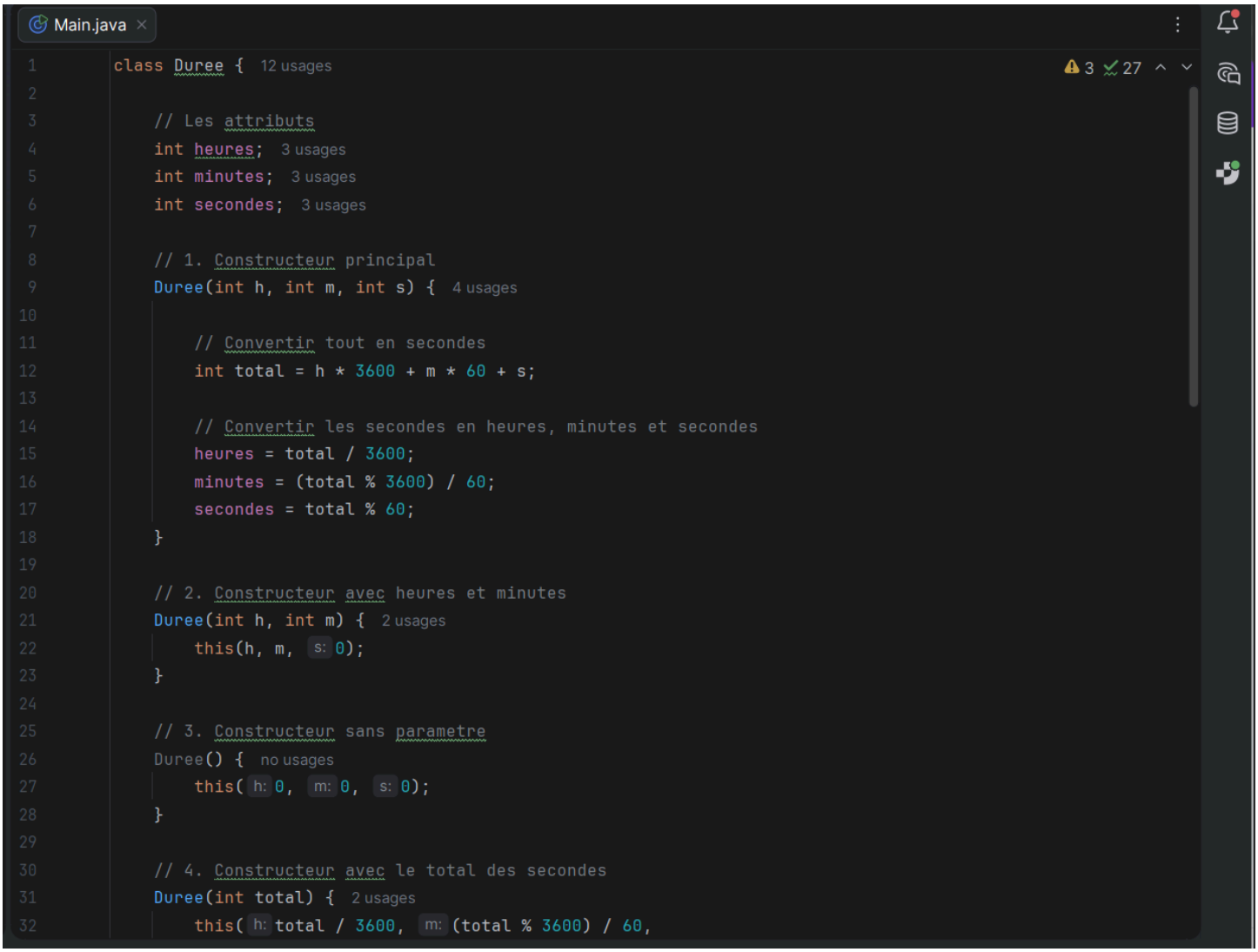
***Objet 2 new Point(2, 3) pour p3 Accessible**

***Objet 3 new Point(0, 0) dans reinitialiser() Inaccessible**

L'objet 1 reste accessible grâce aux références p1 et p2.L'objet 2 reste accessible grâce à la référence p3. L'objet 3 n'est plus accessible après la fin de la méthode reinitialiser(), car la variable locale p disparaît.

Resultat du code :

EXO N 4 :



```java
class Duree {  12 usages
    // 4. Constructeur avec le total des secondes
    Duree(int total) {  2 usages
        this( h: total / 3600,  m: (total % 3600) / 60,
                s: total % 60);
    }

    // Methode pour convertir en secondes
    int enSecondes() {  2 usages
        return heures * 3600 + minutes * 60 + secondes;
    }

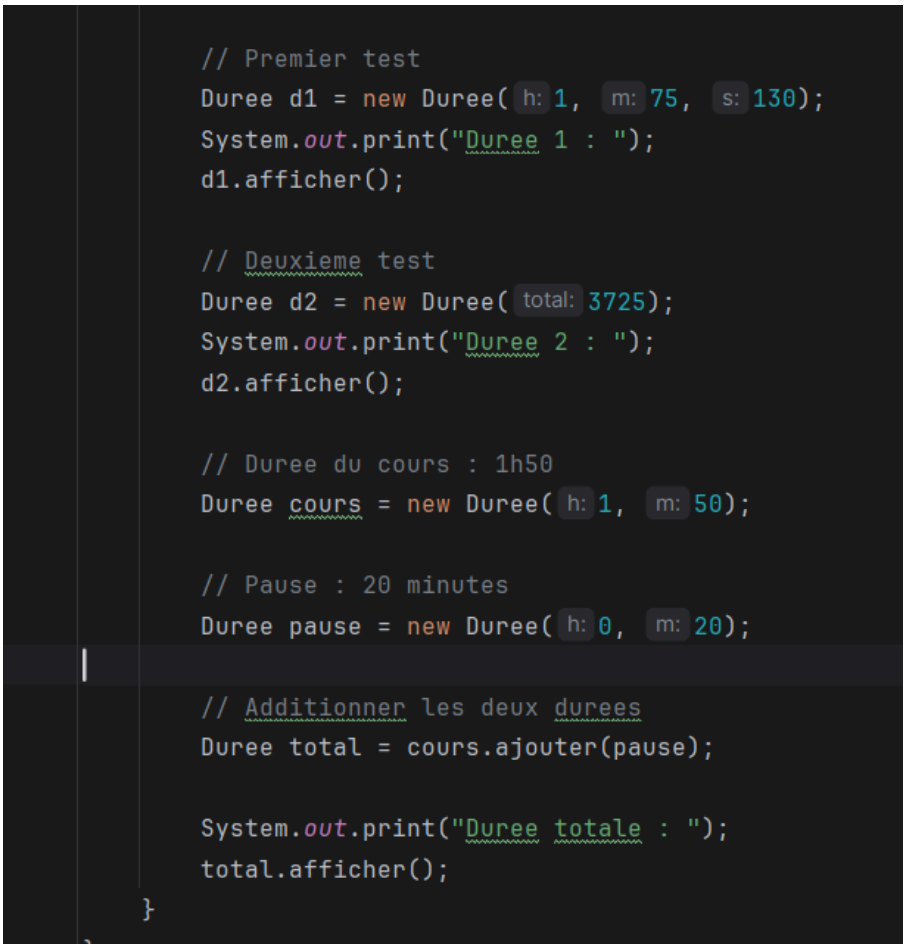
    // Methode pour additionner deux durees
@   Duree ajouter(Duree autre) {  1 usage
        return new Duree( total: enSecondes() + autre.enSecondes());
    }

    // Methode pour afficher la duree
    void afficher() {  3 usages
        System.out.printf("%02dh%02dm%02ds%n",
                heures, minutes, secondes);
    }
}

public class Main {

    public static void main(String[] args) {

        // Premier test
        Duree d1 = new Duree( h: 1,  m: 75,  s: 130);
        System.out.print("Duree 1 : ");
        d1.afficher();
```



```
        // Premier test
        Duree d1 = new Duree( h: 1,  m: 75,  s: 130);
        System.out.print("Duree 1 : ");
        d1.afficher();

        // Deuxieme test
        Duree d2 = new Duree( total: 3725);
        System.out.print("Duree 2 : ");
        d2.afficher();

        // Duree du cours : 1h50
        Duree cours = new Duree( h: 1,  m: 50);

        // Pause : 20 minutes
        Duree pause = new Duree( h: 0,  m: 20);

        // Additionner les deux durees
        Duree total = cours.ajouter(pause);

        System.out.print("Duree totale : ");
        total.afficher();
    }
```

5. Question de réflexion :

Non, ces deux constructeurs ne peuvent pas coexister, car ils possèdent la même signature : un seul paramètre de type int.

Le compilateur Java ne distingue pas les constructeurs par le nom de leurs paramètres.

Solution proposée : utiliser une méthode statique pour créer une durée en heures :

static Duree depuisHeures(int heures) {

return new Duree(heures, 0, 0);

}

EXO 5: Point, Rectangle, Cercle

Main.java

```java
class Point {  19 usages
    // Attributs
    double x;  15 usages
    double y;  12 usages

    // Constructeur
    Point(double x, double y) {  8 usages
        this.x = x;
        this.y = y;
    }

    // Calculer la distance entre deux points
    double distance(Point autre) {  1 usage
        return Math.sqrt(
                (x - autre.x) * (x - autre.x)
                        + (y - autre.y) * (y - autre.y)
        );
    }
}

class Rectangle {  2 usages

    // Attributs
    Point coinHautGauche;  11 usages
    double largeur;  5 usages
    double hauteur;  5 usages

    // Constructeur
    Rectangle(Point coin, double largeur, double hauteur) {  1 usage
        this.coinHautGauche = coin;
        this.largeur = largeur;
```

Main.java

```java
class Rectangle {  2 usages

    // Constructeur
    Rectangle(Point coin, double largeur, double hauteur) {  1 usage
        this.coinHautGauche = coin;
        this.largeur = largeur;
        this.hauteur = hauteur;
    }

    // Calculer la surface
    double surface() {  no usages
        return largeur * hauteur;
    }

    // Calculer le perimetre
    double perimetre() {  no usages
        return 2 * (largeur + hauteur);
    }

    // Retourner le coin en bas a droite
    Point coinBasDroit() {  no usages
        return new Point(
                x: coinHautGauche.x + largeur,
                y: coinHautGauche.y + hauteur
        );
    }

    // Verifier si un point est dans le rectangle
    boolean contient(Point p) {  1 usage
        return p.x >= coinHautGauche.x
                && p.x <= coinHautGauche.x + largeur
                && p.y >= coinHautGauche.y
```

Main.java

```java
class Rectangle {  2 usages
    // Verifier si un point est dans le rectangle
    boolean contient(Point p) {  1 usage
        return p.x >= coinHautGauche.x
                && p.x <= coinHautGauche.x + largeur
                && p.y >= coinHautGauche.y
                && p.y <= coinHautGauche.y + hauteur;
    }

    // Deplacer le rectangle
    void translater(double dx, double dy) {  no usages
        coinHautGauche.x += dx;
        coinHautGauche.y += dy;
    }
}

class Cercle {  2 usages

    // Attributs
    Point centre;  2 usages
    double rayon;  4 usages

    // Constructeur
    Cercle(Point centre, double rayon) {  1 usage
        this.centre = centre;
        this.rayon = rayon;
    }

    // Calculer la surface
    double surface() {  no usages
        return Math.PI * rayon * rayon;
    }
```

Main.java

```java
}

public class Main {

    public static void main(String[] args) {

        // Creer le rectangle
        Point p = new Point( x: 1,  y: 1);
        Rectangle R = new Rectangle(p,  largeur: 4,  hauteur: 3);

        // Creer le cercle
        Cercle C = new Cercle(new Point( x: 0,  y: 0),  rayon: 5);

        // Points a tester
        Point[] points = {
                new Point( x: 2,  y: 2),
                new Point( x: 5,  y: 4),
                new Point( x: 6,  y: 1),
                new Point( x: 3,  y: 4),
                new Point( x: 0,  y: 5)
        };

        // Afficher les resultats
        for (Point point : points) {

            System.out.println(
                    "(" + point.x + "," + point.y + ")"
            );

            System.out.println("Dans R : " + R.contient(point));

            System.out.println("Dans C : " + C.contient(point));
```

```
            System.out.println(
                    "(" + point.x + "," + point.y + ")"
            );

            System.out.println("Dans R : " + R.contient(point));

            System.out.println("Dans C : " + C.contient(point));

            System.out.println("----------------");
        }

        // Tester le piege de reference
        System.out.println("Avant modification : "
                + R.coinHautGauche.x);

        p.x = 100;

        System.out.println("Apres modification : "
                + R.coinHautGauche.x);
    }
}
```

5. Question du piège à observer

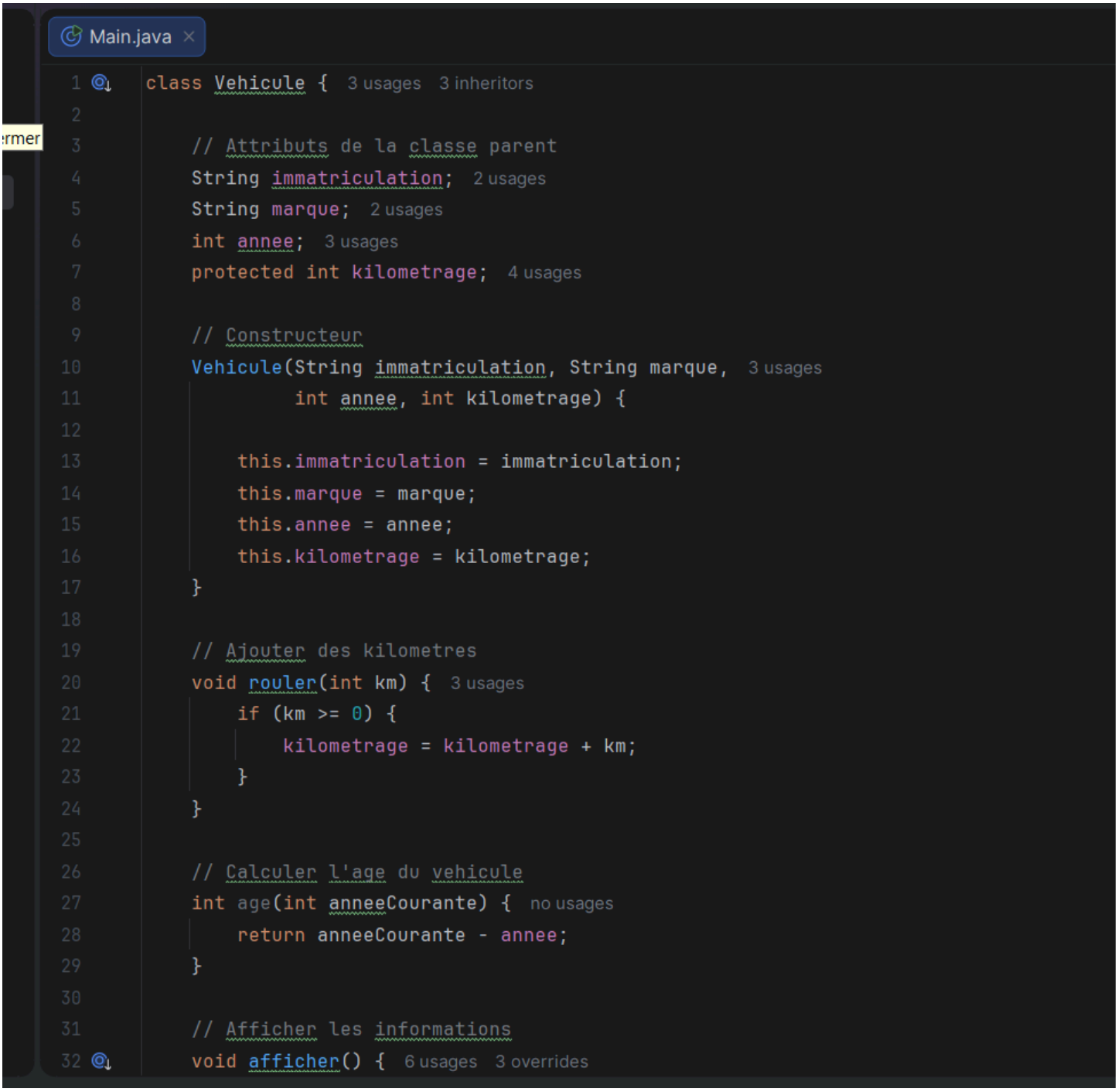
Le rectangle utilise directement la référence de l'objet p. Ainsi, lorsque nous modifions p.x, nous modifions également le coin supérieur gauche du rectangle.

Pour éviter ce comportement, il faut modifier le constructeur de Rectangle :

Au lieu de : this.coinHautGauche = coin;

Écrire :this.coinHautGauche = new Point(coin.x, coin.y);

Exercice 6 : Le parc de véhicules – Héritage en Java.



Main.java

```
1  class Vehicule {  3 usages  3 inheritors
2
3      // Attributs de la classe parent
4      String immatriculation;  2 usages
5      String marque;  2 usages
6      int annee;  3 usages
7      protected int kilometrage;  4 usages
8
9      // Constructeur
10     Vehicule(String immatriculation, String marque,  3 usages
11             int annee, int kilometrage) {
12
13         this.immatriculation = immatriculation;
14         this.marque = marque;
15         this.annee = annee;
16         this.kilometrage = kilometrage;
17     }
18
19     // Ajouter des kilometres
20     void rouler(int km) {  3 usages
21         if (km >= 0) {
22             kilometrage = kilometrage + km;
23         }
24     }
25
26     // Calculer l'age du vehicule
27     int age(int anneeCourante) {  no usages
28         return anneeCourante - annee;
29     }
30
31     // Afficher les informations
32     void afficher() {  6 usages  3 overrides
```

Main.java

```java
class Vehicule {  3 usages  3 inheritors

    // Afficher les informations
    void afficher() {  6 usages  3 overrides
        System.out.println("Immatriculation : " + immatriculation);
        System.out.println("Marque : " + marque);
        System.out.println("Annee : " + annee);
        System.out.println("Kilometrage : " + kilometrage);
    }

    // Vignette du vehicule
    double calculerVignette() {  3 usages  3 overrides
        return 500;
    }
}

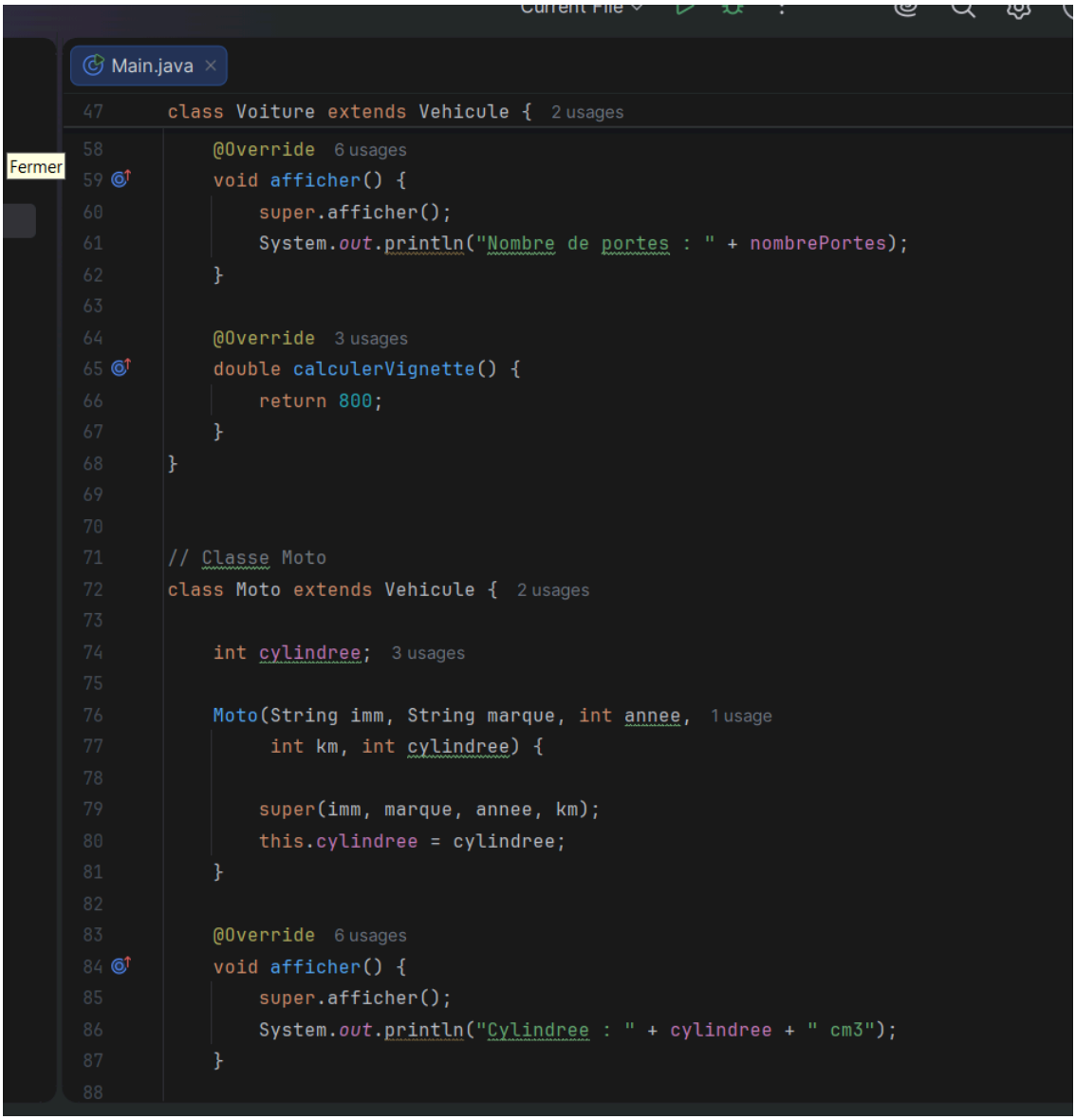

// Classe Voiture
class Voiture extends Vehicule {  2 usages

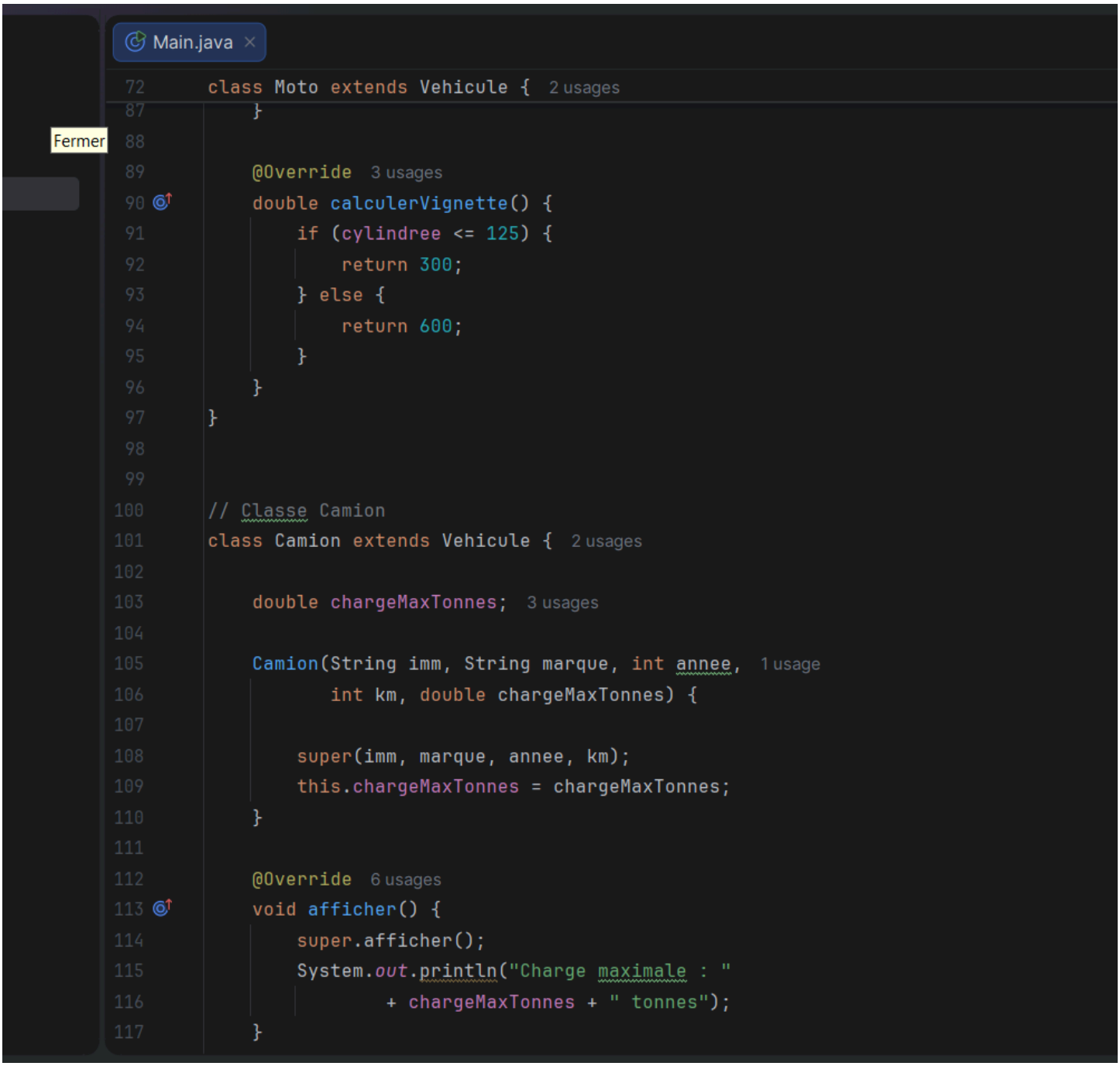
    int nombrePortes;  2 usages

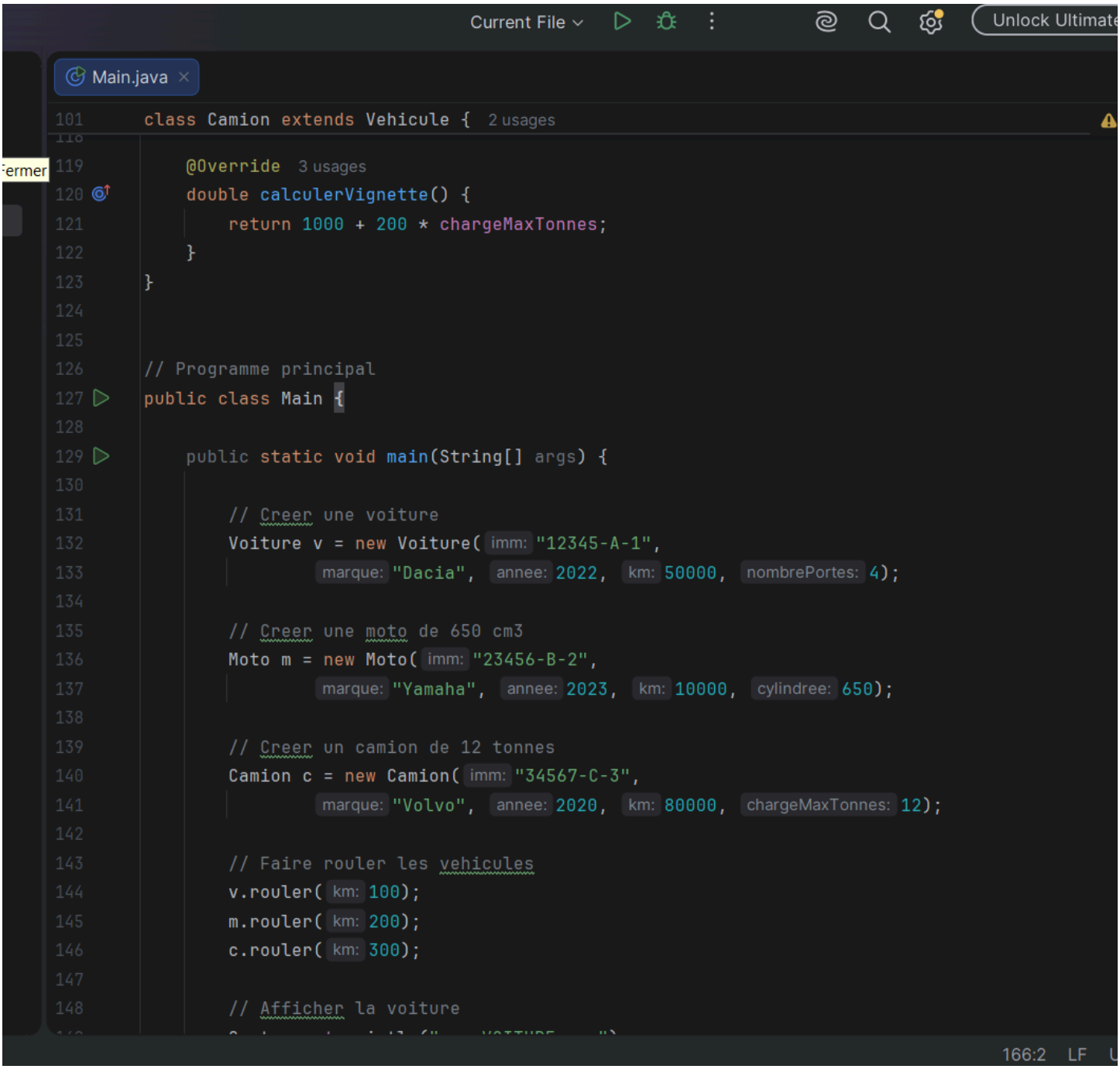
    Voiture(String imm, String marque, int annee,  1 usage
            int km, int nombrePortes) {

        super(imm, marque, annee, km);
        this.nombrePortes = nombrePortes;
    }

    @Override  6 usages
    void afficher() {
        super.afficher();
```


Main.java
Fermer
class Voiture extends Vehicule { 2 usages
@Override 6 usages
void afficher() {
super.afficher();
System.out.println("Nombre de portes : " + nombrePortes);
}
@Override 3 usages
double calculerVignette() {
return 800;
}
}
// Classe Moto
class Moto extends Vehicule { 2 usages
int cylindree; 3 usages
Moto(String imm, String marque, int annee, 1 usage
int km, int cylindree) {
super(imm, marque, annee, km);
this.cylindree = cylindree;
}
@Override 6 usages
void afficher() {
super.afficher();
System.out.println("Cylindree : " + cylindree + " cm3");
}

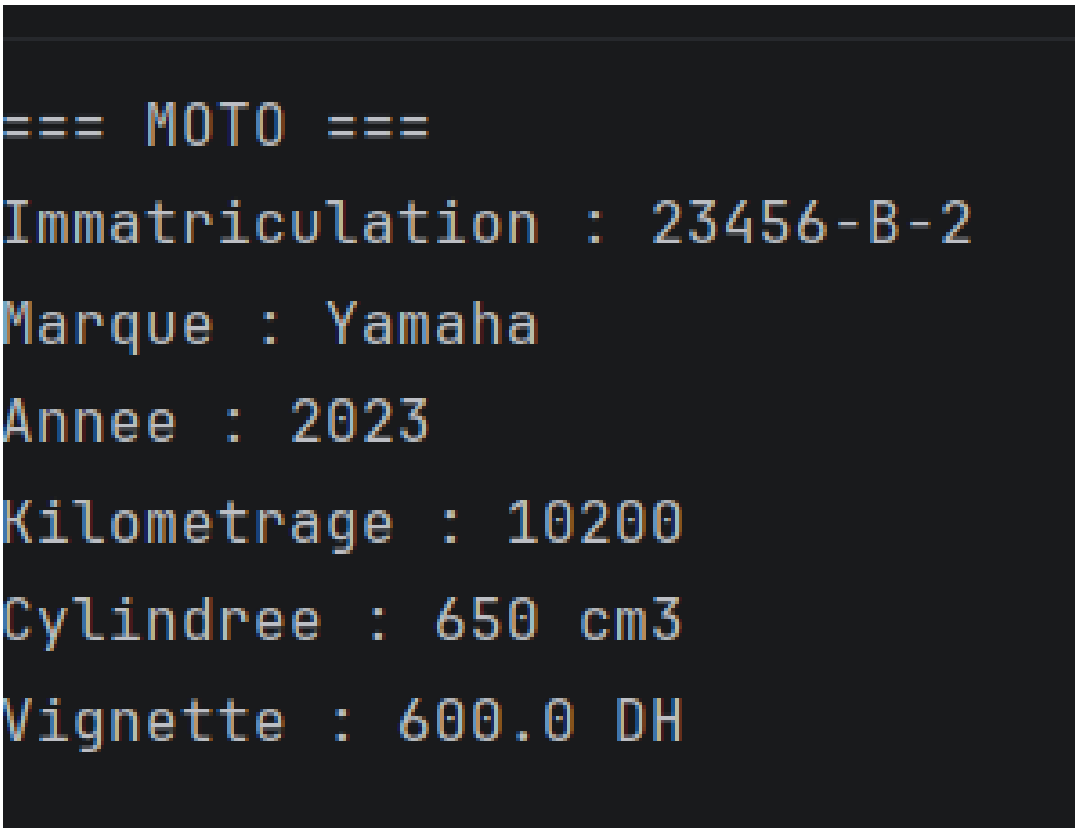

Main.java
Fermer
72 class Moto extends Vehicule { 2 usages
87 }
88
89 @Override 3 usages
90 double calculerVignette() {
91 if (cylindree <= 125) {
92 return 300;
93 } else {
94 return 600;
95 }
96 }
97 }
98
99
100 // Classe Camion
101 class Camion extends Vehicule { 2 usages
102
103 double chargeMaxTonnes; 3 usages
104
105 Camion(String imm, String marque, int annee, 1 usage
106 int km, double chargeMaxTonnes) {
107
108 super(imm, marque, annee, km);
109 this.chargeMaxTonnes = chargeMaxTonnes;
110 }
111
112 @Override 6 usages
113 void afficher() {
114 super.afficher();
115 System.out.println("Charge maximale : "
116 + chargeMaxTonnes + " tonnes");
117 }

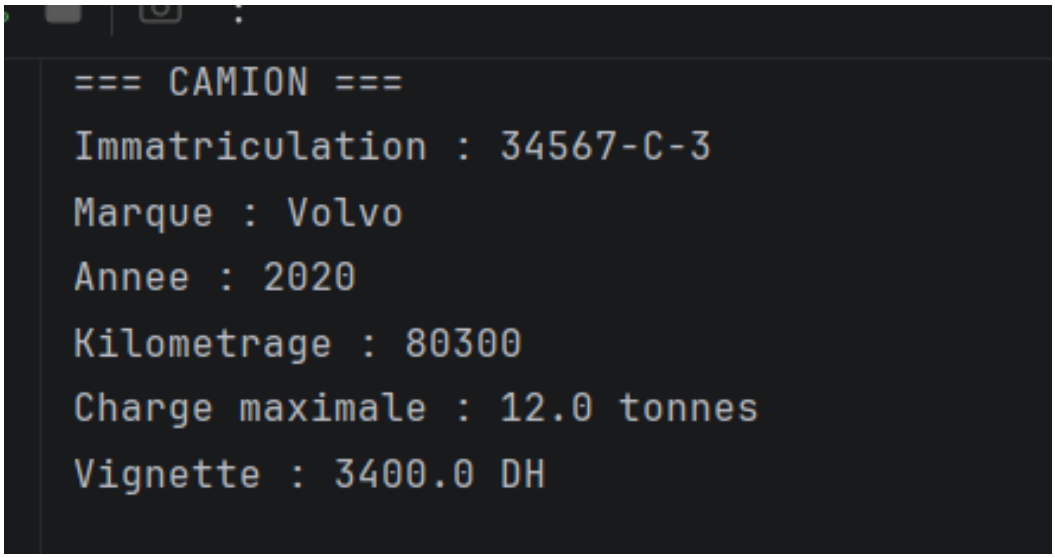

Current File
Unlock Ultimate
Main.java
fermer
class Camion extends Vehicule { 2 usages
@Override 3 usages
double calculerVignette() {
return 1000 + 200 * chargeMaxTonnes;
}
}
// Programme principal
public class Main {
public static void main(String[] args) {
// Creer une voiture
Voiture v = new Voiture(imm: "12345-A-1",
marque: "Dacia", annee: 2022, km: 50000, nombrePortes: 4);
// Creer une moto de 650 cm3
Moto m = new Moto(imm: "23456-B-2",
marque: "Yamaha", annee: 2023, km: 10000, cylindree: 650);
// Creer un camion de 12 tonnes
Camion c = new Camion(imm: "34567-C-3",
marque: "Volvo", annee: 2020, km: 80000, chargeMaxTonnes: 12);
// Faire rouler les vehicules
v.rouler(km: 100);
m.rouler(km: 200);
c.rouler(km: 300);
// Afficher la voiture
166:2
LF

Main.java

```
27      public class Main {
29          public static void main(String[] args) {
41                  marque: "Volvo",  annee: 2020,  km: 80000,  chargeMaxTonnes: 12);
42
43              // Faire rouler les vehicules
44              v.rouler( km: 100);
45              m.rouler( km: 200);
46              c.rouler( km: 300);
47
48              // Afficher la voiture
49              System.out.println("=== VOITURE ===");
50              v.afficher();
51              System.out.println("Vignette : "
52                      + v.calculerVignette() + " DH");
53
54              // Afficher la moto
55              System.out.println("\n=== MOTO ===");
56              m.afficher();
57              System.out.println("Vignette : "
58                      + m.calculerVignette() + " DH");
59
60              // Afficher le camion
61              System.out.println("\n=== CAMION ===");
62              c.afficher();
63              System.out.println("Vignette : "
64                      + c.calculerVignette() + " DH");
65          }
66      }
```

```
C:\Users\aelki\.jdks\openjdk-26.0.2.1\bin\jav
=== VOITURE ===
Immatriculation : 12345-A-1
Marque : Dacia
Annee : 2022
Kilometrage : 50100
Nombre de portes : 4
Vignette : 800.0 DH
```


=== MOTO ===
Immatriculation : 23456-B-2
Marque : Yamaha
Annee : 2023
Kilometrage : 10200
Cylindree : 650 cm3
Vignette : 600.0 DH


=== CAMION ===
Immatriculation : 34567-C-3
Marque : Volvo
Annee : 2020
Kilometrage : 80300
Charge maximale : 12.0 tonnes
Vignette : 3400.0 DH

```
=== CAMION ===
Immatriculation : 34567-C-3
Marque : Volvo
Annee : 2020
Kilometrage : 80300
Charge maximale : 12.0 tonnes
Vignette : 3400.0 DH
```

Expériences guidées:

1-Si tu déclares kilometrage en private, la classe Camion ne peut pas y accéder directement.

Avec protected, les classes enfants peuvent accéder à cet attribut.

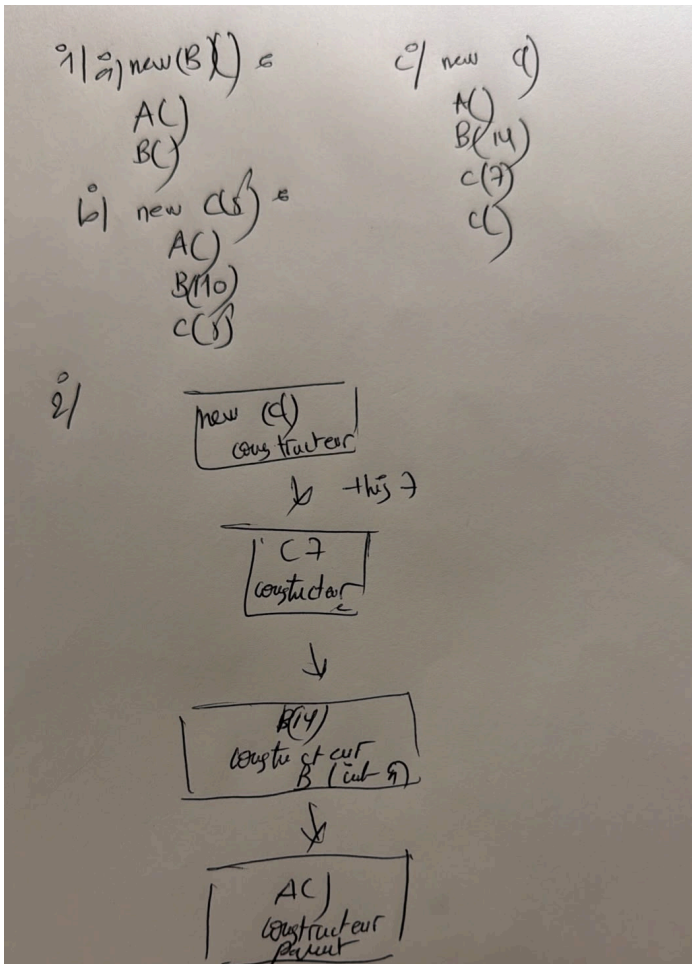
2-@Override

```
double calculerVignete() {
    return 600;
}
```

Le compilateur affiche une erreur, car le nom de la méthode ne correspond pas à celui de la classe parent.

Si tu retires @Override, le programme peut compiler, mais la méthode ne redéfinit plus calculerVignette(). C'est pourquoi l'annotation est importante.

EXO N 7 :


a) new B()
A()
B()
b) new C(5)
A()
B(10)
C(5)
c) new C()
A()
B(14)
C(7)
C()
2)
new C()
constructeur
this
C7
constructeur
B(14)
constructeur B(int 7)
AC
constructeur parent

Main.java

```java
class A {  1 usage  2 inheritors
    A() {  2 usages
        System.out.println("A()");
    }
}

class B extends A {  2 usages  1 inheritor
    B() {  1 usage
        System.out.println("B()");
    }

    B(int n) {  1 usage
        System.out.println("B(" + n + ")");
    }
}

class C extends B {  2 usages
    C() {  1 usage
        this( n: 7);
        System.out.println("C()");
    }

    C(int n) {  2 usages
        super( n: n * 2);
        System.out.println("C(" + n + ")");
    }
}

public class Main {
    public static void main(String[] args) {

        System.out.println("Test 1");
```

Main.java

```java
}

class C extends B {  2 usages
    C() {  1 usage
        this( n: 7);
        System.out.println("C()");
    }

    C(int n) {  2 usages
        super( n: n * 2);
        System.out.println("C(" + n + ")");
    }
}

public class Main {
    public static void main(String[] args) {

        System.out.println("Test 1");
        new B();

        System.out.println("Test 2");
        new C( n: 5);

        System.out.println("Test 3");
        new C();
    }
}
```

Main.java

```java
class A {  1 usage  2 inheritors
    A(String nom) {  2 usages
        System.out.println("A(" + nom + ")");
    }
}
class B extends A {  2 usages  1 inheritor

    B() {  1 usage
        super( nom: "Parent");
        System.out.println("B()");
    }

    B(int n) {  1 usage
        super( nom: "Parent");
        System.out.println("B(" + n + ")");
    }
}
```

Parce que les constructeurs de B appellent automatiquement super() lorsqu'ils ne précisent pas de constructeur parent.

Mais maintenant, A() n'existe plus. Il faut donc appeler A(String nom).

Correction minimale :

<u>Dans la classe B :</u>

```java
    B() {  1 usage
        super( nom: "Parent");
        System.out.println("B()");
    }

    B(int n) {  1 usage
        super( nom: "Parent");
        System.out.println("B(" + n + ")");
    }
```

5-Non, on ne peut pas utiliser this(...) et super(...) dans le même constructeur, car les deux instructions doivent être placées obligatoirement en première position du constructeur.

On ne peut donc pas les utiliser ensemble.

Exemple incorrect :

```
C() {
   this(7);
   super(14); // Erreur
}
```

Exemple correct :

```
C() {
   this(7);
}
C(int n) {
   super(n * 2);
   System.out.println("C(" + n + ")");
}
```

Conclusion : **this() et super() permettent de gérer l'ordre d'appel des constructeurs, mais un constructeur ne peut utiliser que l'un des deux directement.**

***EXO N 8 :*Chasse aux erreurs**

N°	Erreur	Explication
1	public void Produit(…)	Ce n'est pas un constructeur, car il contient void.
2	this.datePeremption = date avant super()	super() doit être la première instruction du constructeur.
3	@Override prixTTC(double taux)	La méthode du parent n'a pas de paramètre.
4	return prix * (1 + taux)	prix est private dans Produit, donc inaccessible directement dans la classe enfant.
5	static void afficher() avec this	On ne peut pas utiliser this dans une méthode static.
6	extends ProduitAlimentaire, Produit	Java ne permet pas l'héritage de deux classes.
7	new Produit("Stylo", 5)	Le constructeur demandé n'existe pas à cause de l'erreur n°1.
8	lait.datePeremption	L'attribut est private et inaccessible directement depuis Main.

2. Classer les erreurs

A. Erreurs de compilation :

L'ordre de super().

L'annotation @Override incorrecte.

L'accès à prix private.

L'utilisation de this dans une méthode static.

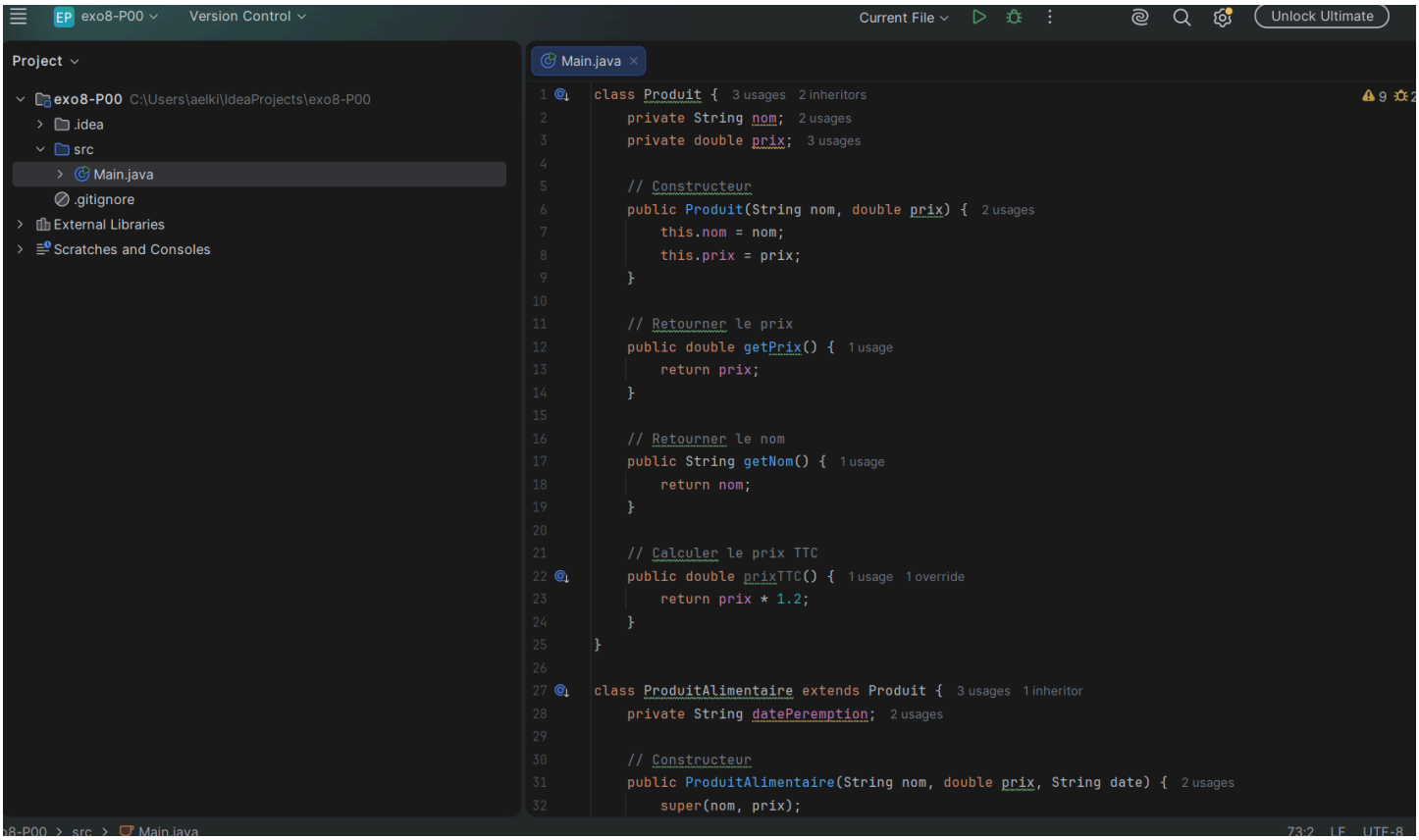
L'héritage multiple.

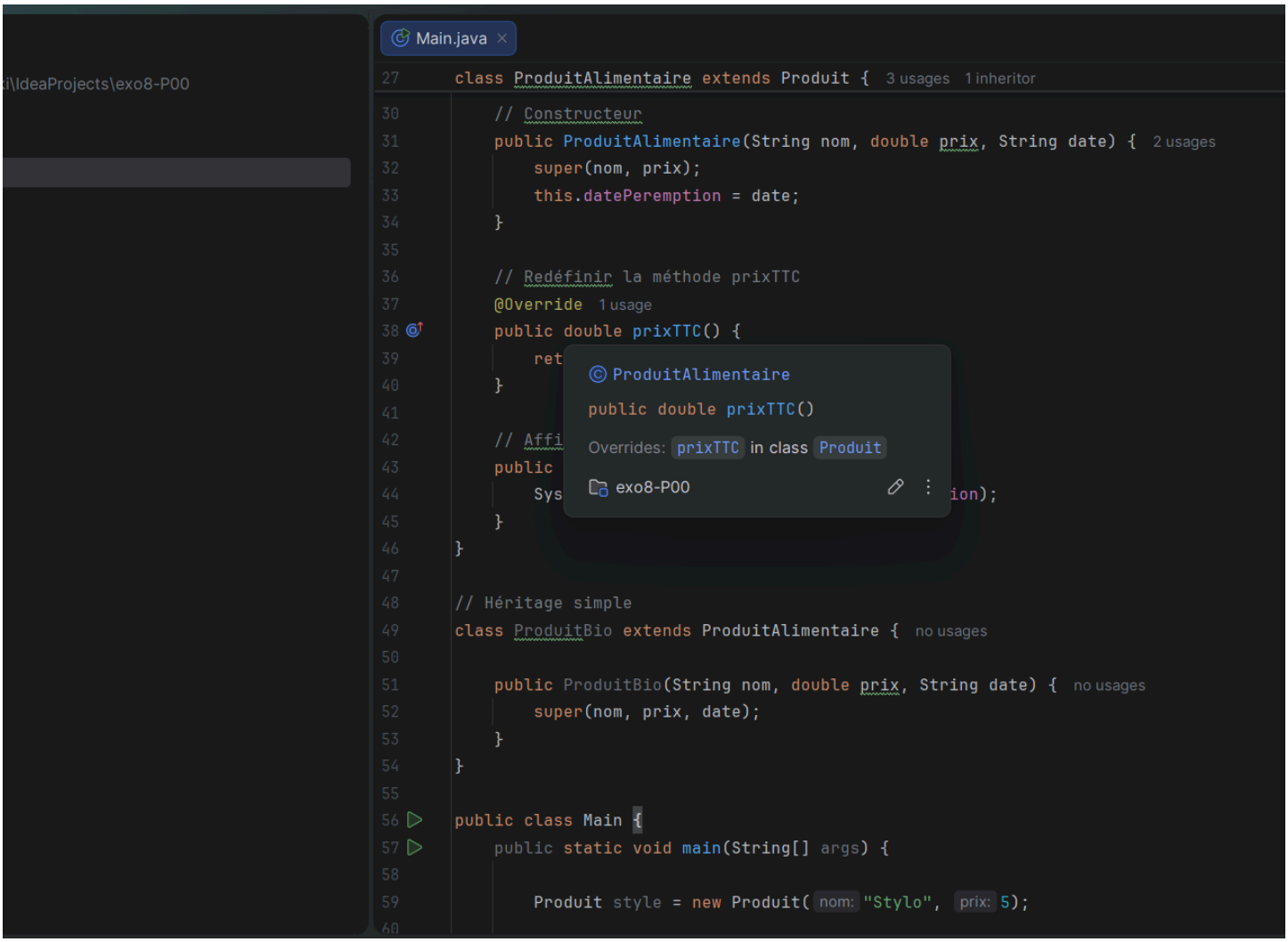
L'appel au constructeur inexistant.

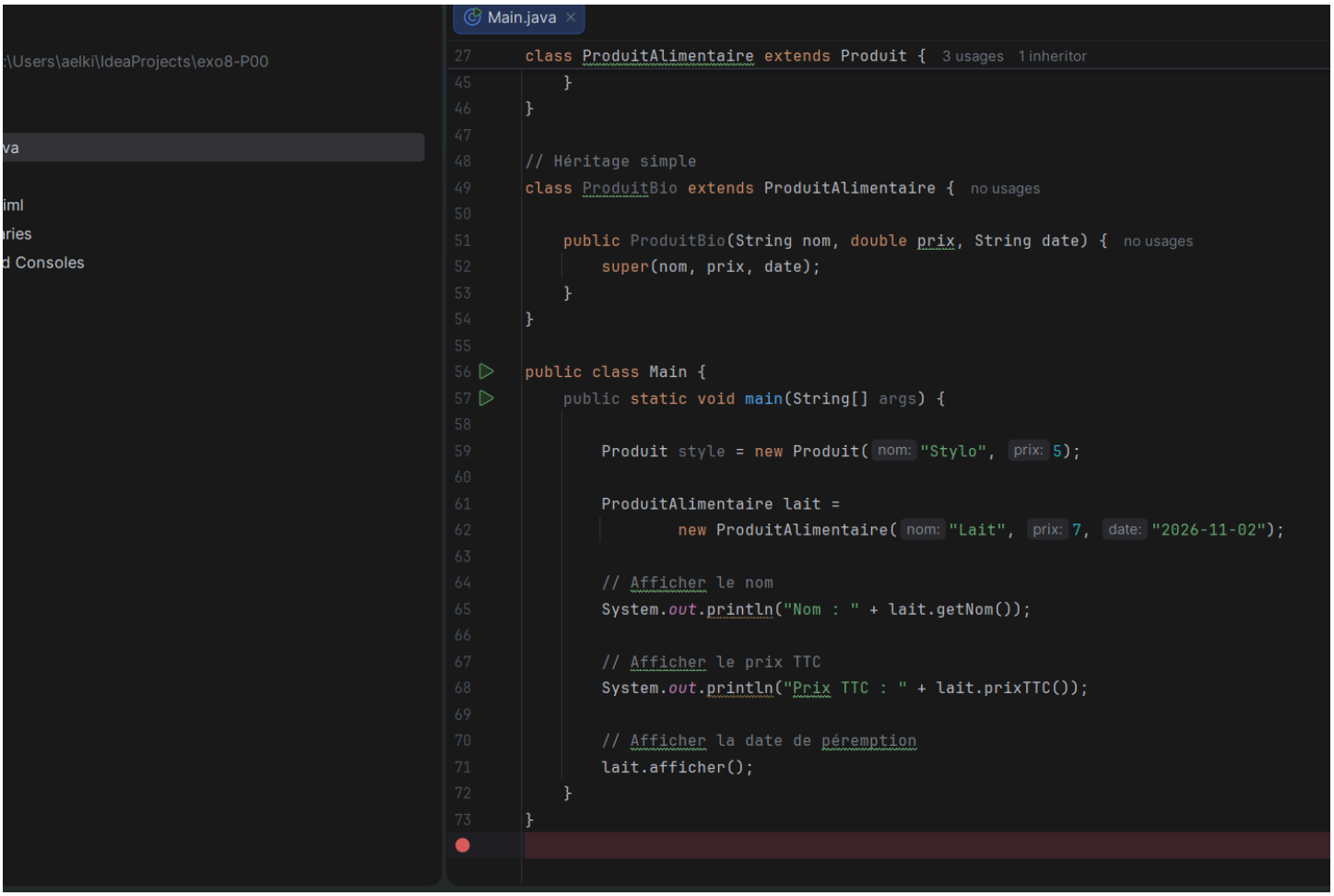
L'accès à datePeremption private.

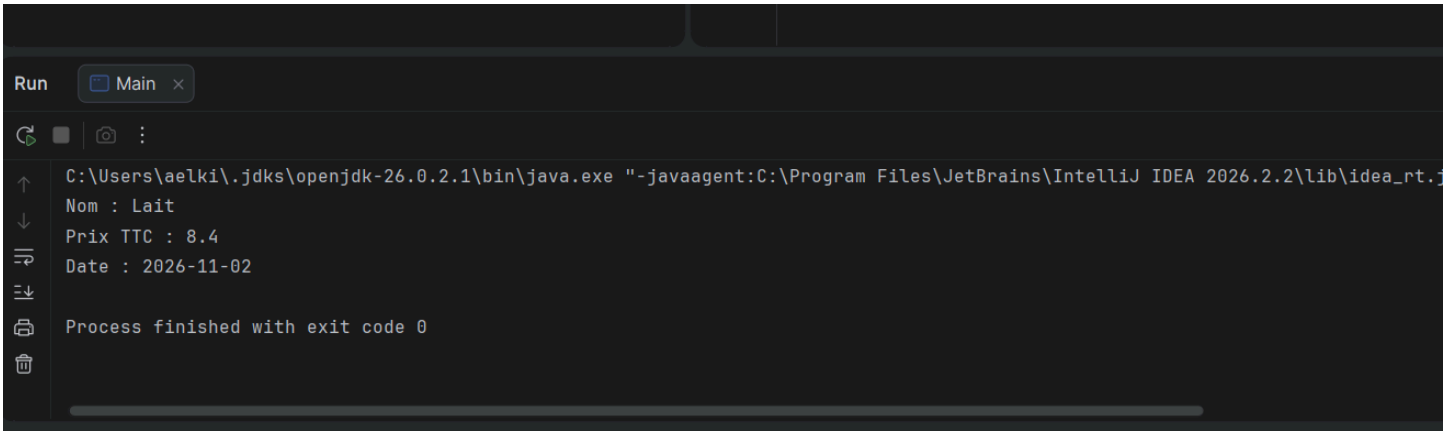
B. Compile, mais faux :

public void Produit(...) : Java considère cette déclaration comme une méthode et non comme un constructeur. Le programme peut donc compiler à cet endroit, mais la création d'un produit avec deux paramètres ne fonctionne pas.

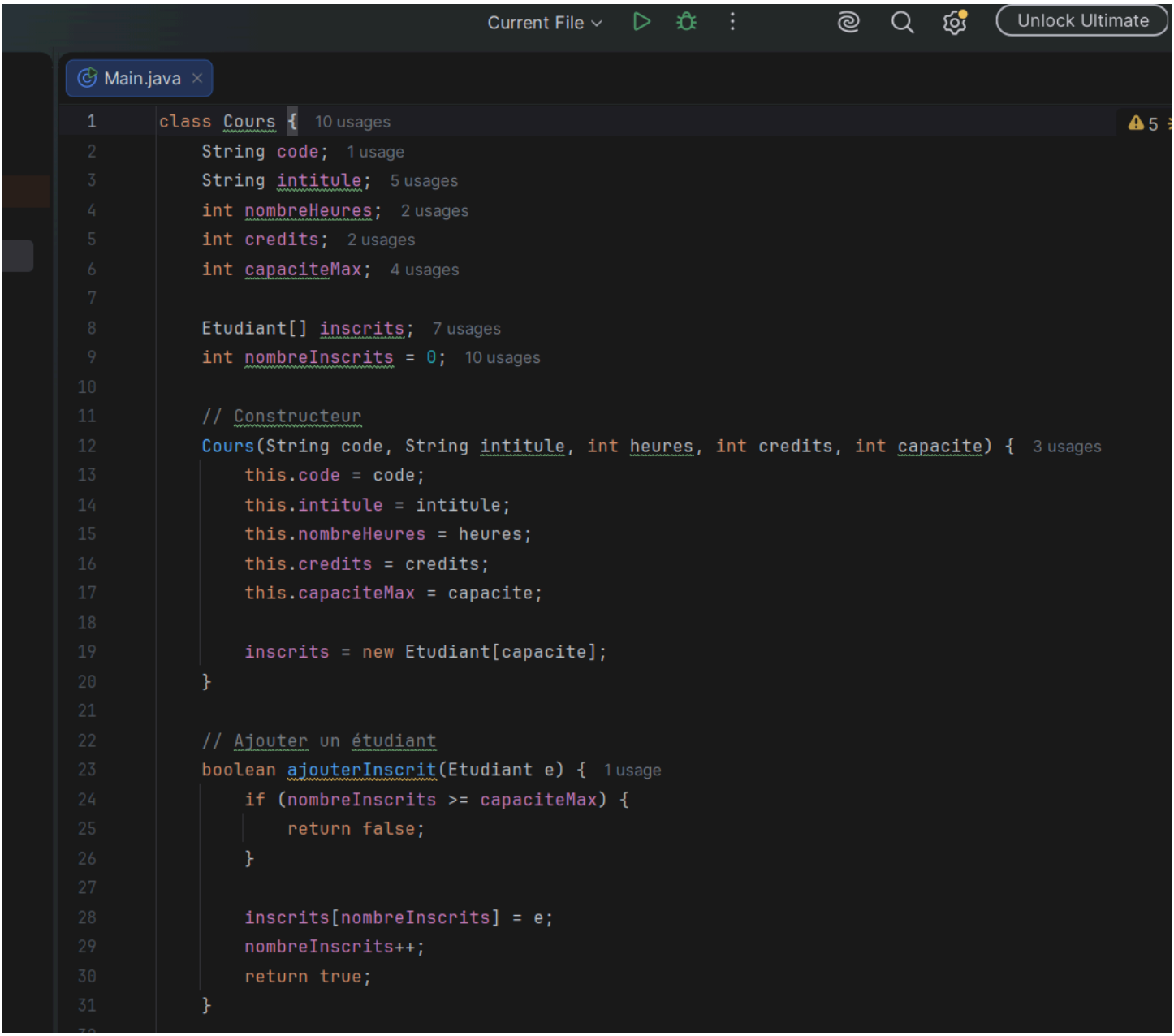

exo8-P00
Version Control
Current File
Unlock Ultimate
Project
exo8-P00 C:\Users\aelki\IdeaProjects\exo8-P00
.idea
src
Main.java
.gitignore
External Libraries
Scratches and Consoles
Main.java
class Produit { 3 usages 2 inheritors
private String nom; 2 usages
private double prix; 3 usages
// Constructeur
public Produit(String nom, double prix) { 2 usages
this.nom = nom;
this.prix = prix;
}
// Retourner le prix
public double getPrix() { 1 usage
return prix;
}
// Retourner le nom
public String getNom() { 1 usage
return nom;
}
// Calculer le prix TTC
public double prixTTC() { 1 usage 1 override
return prix * 1.2;
}
}
class ProduitAlimentaire extends Produit { 3 usages 1 inheritor
private String datePeremption; 2 usages
// Constructeur
public ProduitAlimentaire(String nom, double prix, String date) { 2 usages
super(nom, prix);


Main.java
class ProduitAlimentaire extends Produit { 3 usages 1 inheritor
// Constructeur
public ProduitAlimentaire(String nom, double prix, String date) { 2 usages
super(nom, prix);
this.datePeremption = date;
}
// Redéfinir la méthode prixTTC
@Override 1 usage
public double prixTTC() {
ret
}
ProduitAlimentaire
public double prixTTC()
Overrides: prixTTC in class Produit
exo8-P00
// Affi
public
Sys
ion);
}
}
// Héritage simple
class ProduitBio extends ProduitAlimentaire { no usages
public ProduitBio(String nom, double prix, String date) { no usages
super(nom, prix, date);
}
}
public class Main {
public static void main(String[] args) {
Produit style = new Produit(nom: "Stylo", prix: 5);





EXO 9: INSCRIPTIONS AUX COURS (JAVA)


Current File
Unlock Ultimate
Main.java
class Cours { 10 usages
String code; 1 usage
String intitule; 5 usages
int nombreHeures; 2 usages
int credits; 2 usages
int capaciteMax; 4 usages
Etudiant[] inscrits; 7 usages
int nombreInscrits = 0; 10 usages
// Constructeur
Cours(String code, String intitule, int heures, int credits, int capacite) { 3 usages
this.code = code;
this.intitule = intitule;
this.nombreHeures = heures;
this.credits = credits;
this.capaciteMax = capacite;
inscrits = new Etudiant[capacite];
}
// Ajouter un étudiant
boolean ajouterInscrit(Etudiant e) { 1 usage
if (nombreInscrits >= capaciteMax) {
return false;
}
inscrits[nombreInscrits] = e;
nombreInscrits++;
return true;
}

Main.java

```java
class Cours {  10 usages

    // Supprimer un étudiant
    void retirerInscrit(Etudiant e) {  1 usage
        for (int i = 0; i < nombreInscrits; i++) {
            if (inscrits[i] == e) {

                // Décaler les éléments
                for (int j = i; j < nombreInscrits - 1; j++) {
                    inscrits[j] = inscrits[j + 1];
                }

                nombreInscrits--;
                inscrits[nombreInscrits] = null;
                break;
            }
        }
    }

    // Afficher les étudiants inscrits
    void afficherInscrits() {  3 usages
        System.out.println("Cours : " + intitule);

        for (int i = 0; i < nombreInscrits; i++) {
            System.out.println(inscrits[i].nom);
        }

        double taux = (double) nombreInscrits / capaciteMax * 100;
        System.out.println("Remplissage : " + taux + "%");
    }
}

```

Main.java

```java

class Etudiant {  12 usages
    String numero;  1 usage
    String nom;  5 usages

    Cours[] cours = new Cours[5];  9 usages
    int nombreCours = 0;  9 usages

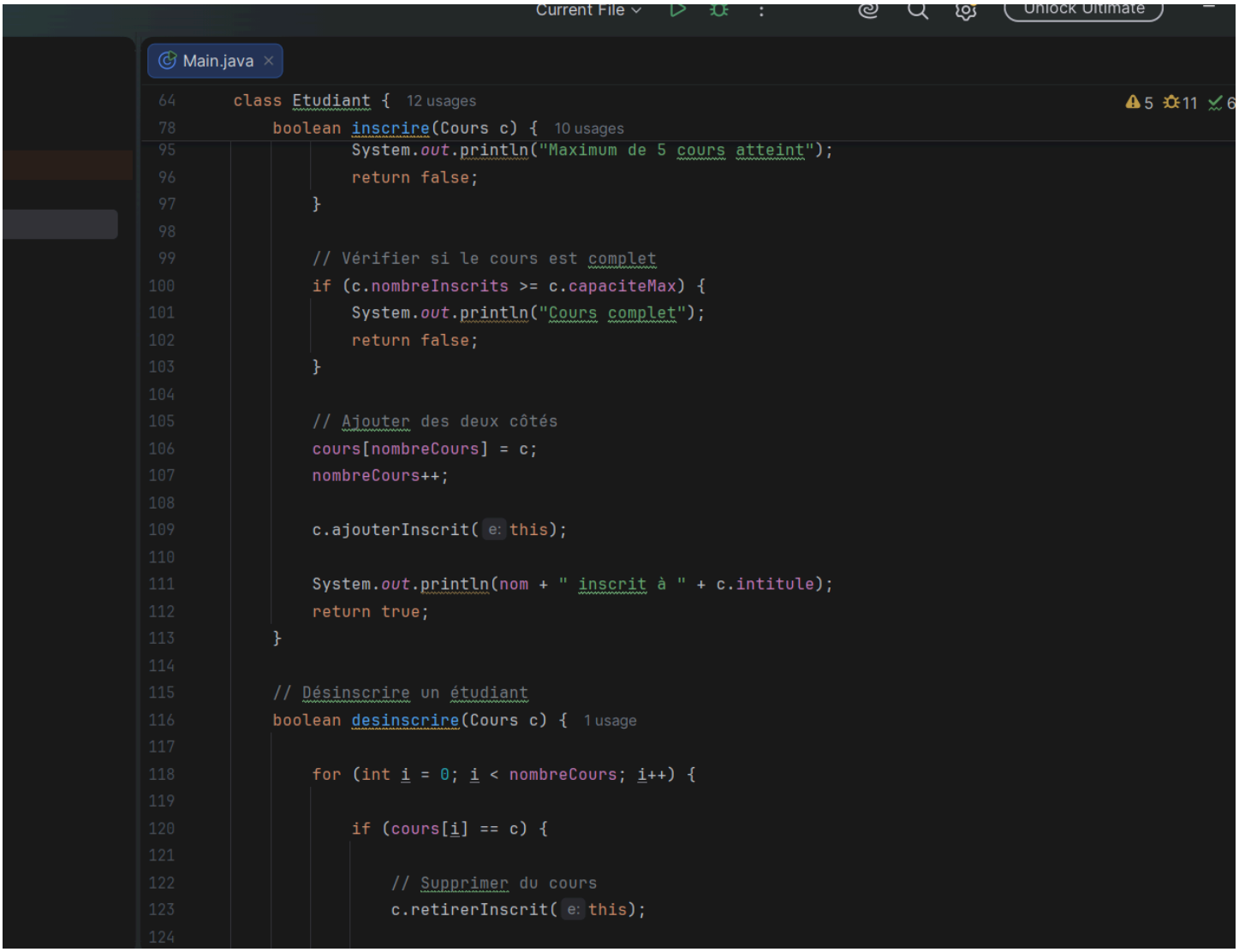
    // Constructeur
    Etudiant(String numero, String nom) {  4 usages
        this.numero = numero;
        this.nom = nom;
    }

    // Inscrire un étudiant à un cours
    boolean inscrire(Cours c) {  10 usages

        if (c == null) {
            System.out.println("Cours invalide");
            return false;
        }

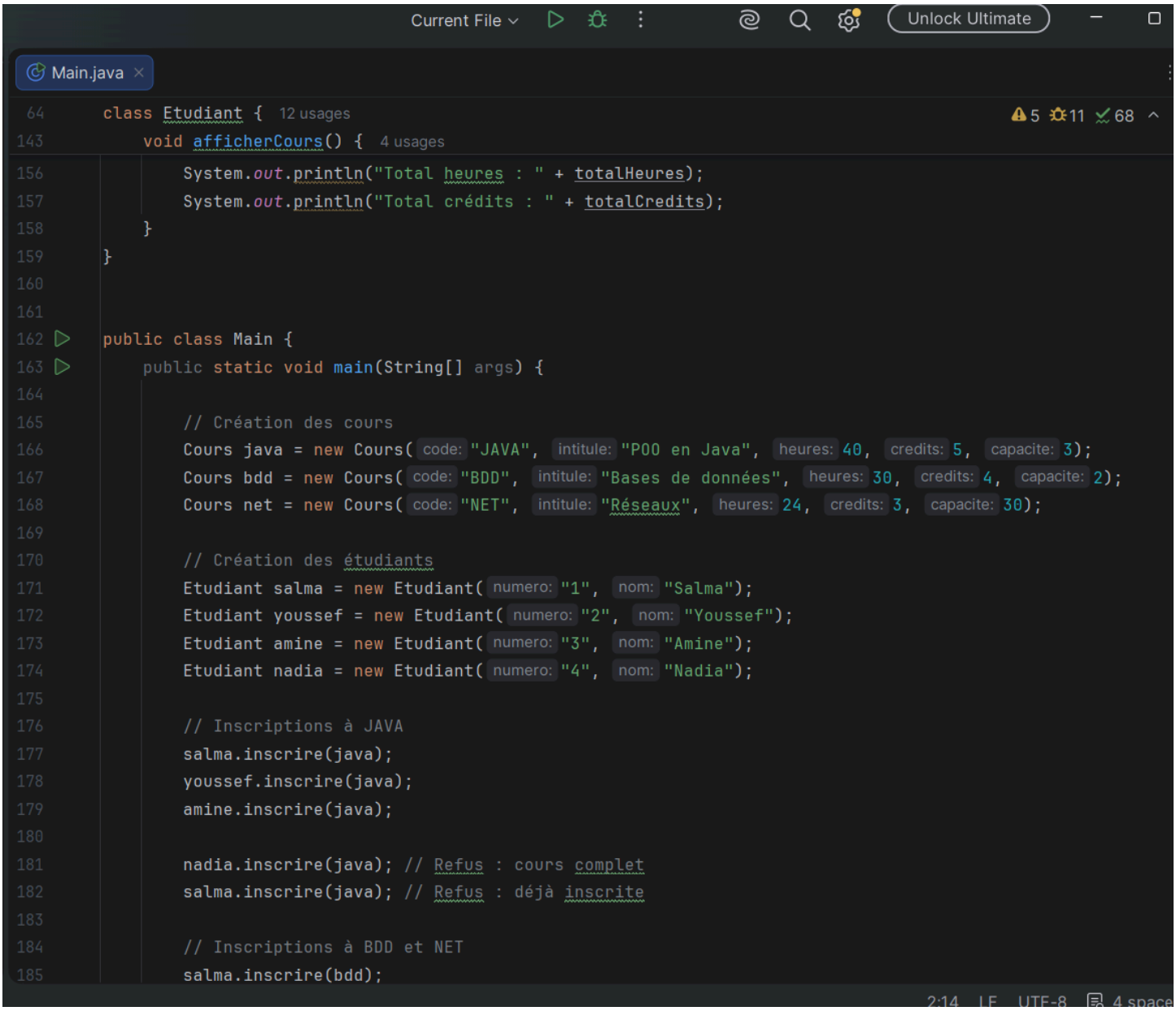
        // Vérifier si déjà inscrit
        for (int i = 0; i < nombreCours; i++) {
            if (cours[i] == c) {
                System.out.println("Déjà inscrit");
                return false;
            }
        }

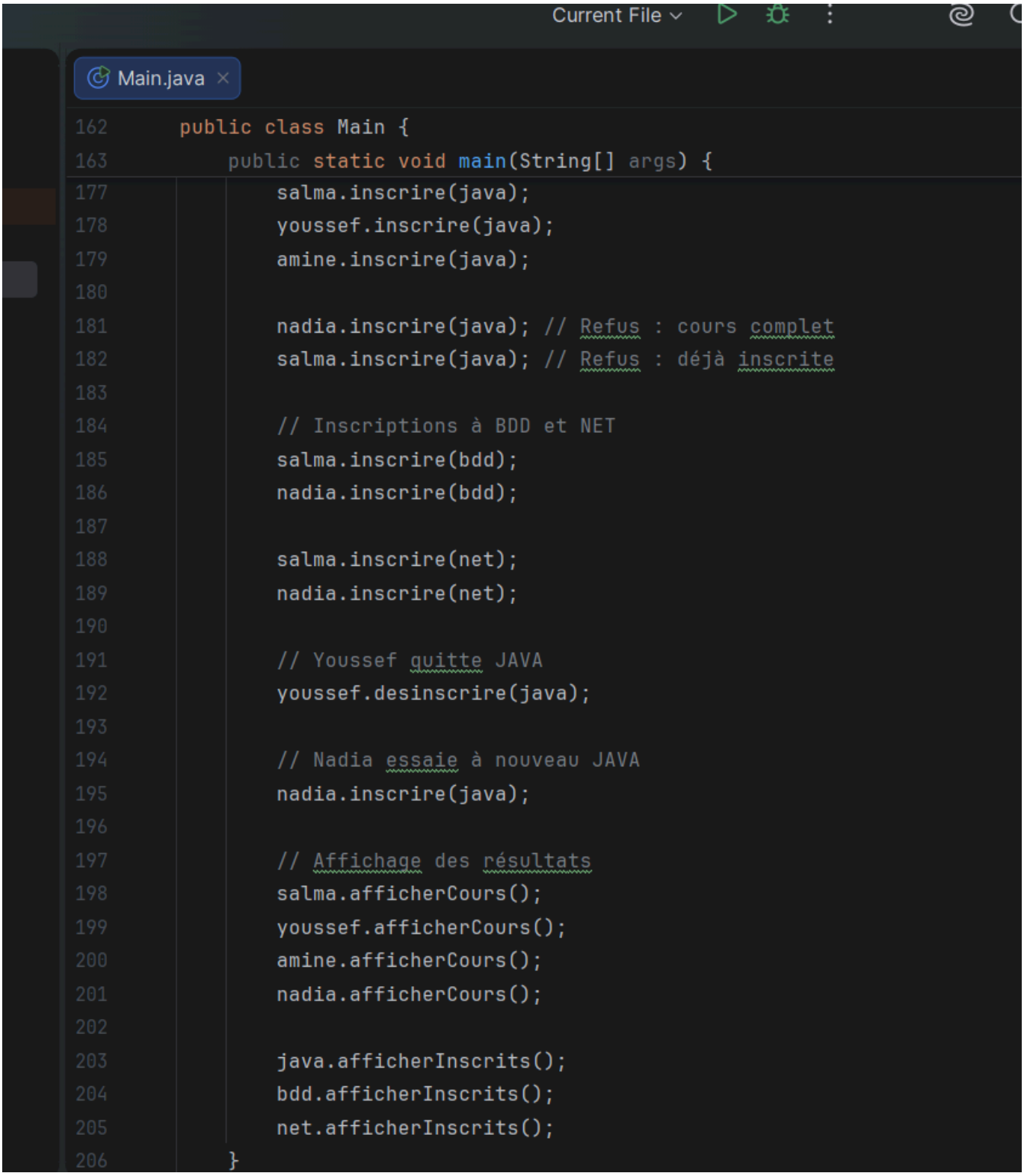
        // Vérifier le nombre de cours
        if (nombreCours >= 5) {
```


Current File
Unlock Ultimate
Main.java
class Etudiant { 12 usages
boolean inscrire(Cours c) { 10 usages
System.out.println("Maximum de 5 cours atteint");
return false;
}
// Vérifier si le cours est complet
if (c.nombreInscrits >= c.capaciteMax) {
System.out.println("Cours complet");
return false;
}
// Ajouter des deux côtés
cours[nombreCours] = c;
nombreCours++;
c.ajouterInscrit(e: this);
System.out.println(nom + " inscrit à " + c.intitule);
return true;
}
// Désinscrire un étudiant
boolean desinscrire(Cours c) { 1 usage
for (int i = 0; i < nombreCours; i++) {
if (cours[i] == c) {
// Supprimer du cours
c.retirerInscrit(e: this);

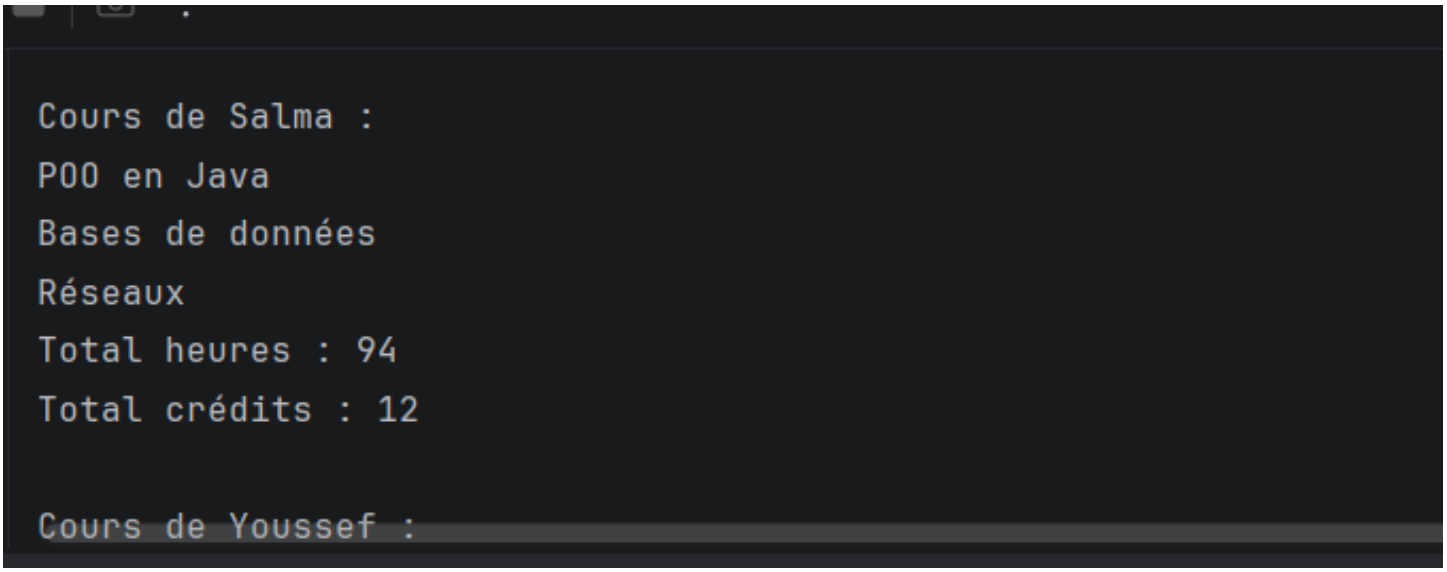
Main.java

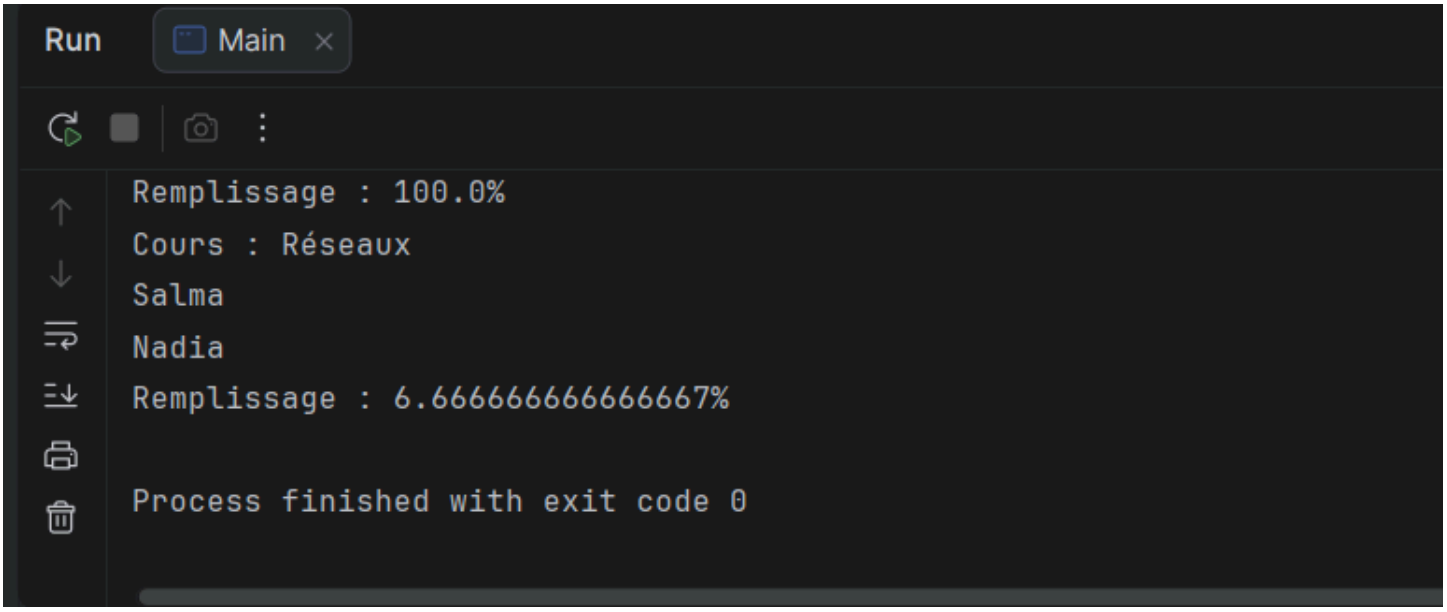
```
64      class Etudiant {  12 usages
116         boolean desinscrire(Cours c) {  1 usage
125                     // Décaler les éléments
126                     for (int j = i; j < nombreCours - 1; j++) {
127                         cours[j] = cours[j + 1];
128                     }
129
130                     nombreCours--;
131                     cours[nombreCours] = null;
132
133                     System.out.println(nom + " désinscrit de " + c.intitule);
134                     return true;
135                 }
136             }
137
138             System.out.println("Étudiant non inscrit à ce cours");
139             return false;
140         }
141
142         // Afficher les cours et le total
143         void afficherCours() {  4 usages
144             int totalHeures = 0;
145             int totalCredits = 0;
146
147             System.out.println("\nCours de " + nom + " :");
148
149             for (int i = 0; i < nombreCours; i++) {
150                 System.out.println(cours[i].intitule);
151
152                 totalHeures += cours[i].nombreHeures;
153                 totalCredits += cours[i].credits;
154             }
```


Current File
Unlock Ultimate
Main.java
class Etudiant { 12 usages
void afficherCours() { 4 usages
5 11 68
System.out.println("Total heures : " + totalHeures);
System.out.println("Total crédits : " + totalCredits);
}
}
public class Main {
public static void main(String[] args) {
// Création des cours
Cours java = new Cours(code: "JAVA", intitule: "POO en Java", heures: 40, credits: 5, capacite: 3);
Cours bdd = new Cours(code: "BDD", intitule: "Bases de données", heures: 30, credits: 4, capacite: 2);
Cours net = new Cours(code: "NET", intitule: "Réseaux", heures: 24, credits: 3, capacite: 30);
// Création des étudiants
Etudiant salma = new Etudiant(numero: "1", nom: "Salma");
Etudiant youssef = new Etudiant(numero: "2", nom: "Youssef");
Etudiant amine = new Etudiant(numero: "3", nom: "Amine");
Etudiant nadia = new Etudiant(numero: "4", nom: "Nadia");
// Inscriptions à JAVA
salma.inscrire(java);
youssef.inscrire(java);
amine.inscrire(java);
nadia.inscrire(java); // Refus : cours complet
salma.inscrire(java); // Refus : déjà inscrite
// Inscriptions à BDD et NET
salma.inscrire(bdd);
2:14 LF UTF-8 4 space


Current File
Main.java
public class Main {
public static void main(String[] args) {
salma.inscrire(java);
youssef.inscrire(java);
amine.inscrire(java);
nadia.inscrire(java); // Refus : cours complet
salma.inscrire(java); // Refus : déjà inscrite
// Inscriptions à BDD et NET
salma.inscrire(bdd);
nadia.inscrire(bdd);
salma.inscrire(net);
nadia.inscrire(net);
// Youssef quitte JAVA
youssef.desinscrire(java);
// Nadia essaie à nouveau JAVA
nadia.inscrire(java);
// Affichage des résultats
salma.afficherCours();
youssef.afficherCours();
amine.afficherCours();
nadia.afficherCours();
java.afficherInscrits();
bdd.afficherInscrits();
net.afficherInscrits();
}


C:\Users\aelki\.jdks\openjdk-26.0.2.1\bin\java.exe "-javaagent:C:\Progra
Salma inscrit à POO en Java
Youssef inscrit à POO en Java
Amine inscrit à POO en Java
Cours complet
Déjà inscrit
Salma inscrit à Bases de données
Nadia inscrit à Bases de données


Cours de Salma :
POO en Java
Bases de données
Réseaux
Total heures : 94
Total crédits : 12
Cours de Youssef :


Run
Main
Remplissage : 100.0%
Cours : Réseaux
Salma
Nadia
Remplissage : 6.666666666666667%
Process finished with exit code 0

Question de réflexion:

Si on met à jour un seul côté, les informations deviennent incohérentes.

Par exemple, si Salma est ajoutée à la liste des cours qu'elle suit, mais qu'elle n'est pas ajoutée à la liste des étudiants inscrits au cours JAVA, alors :

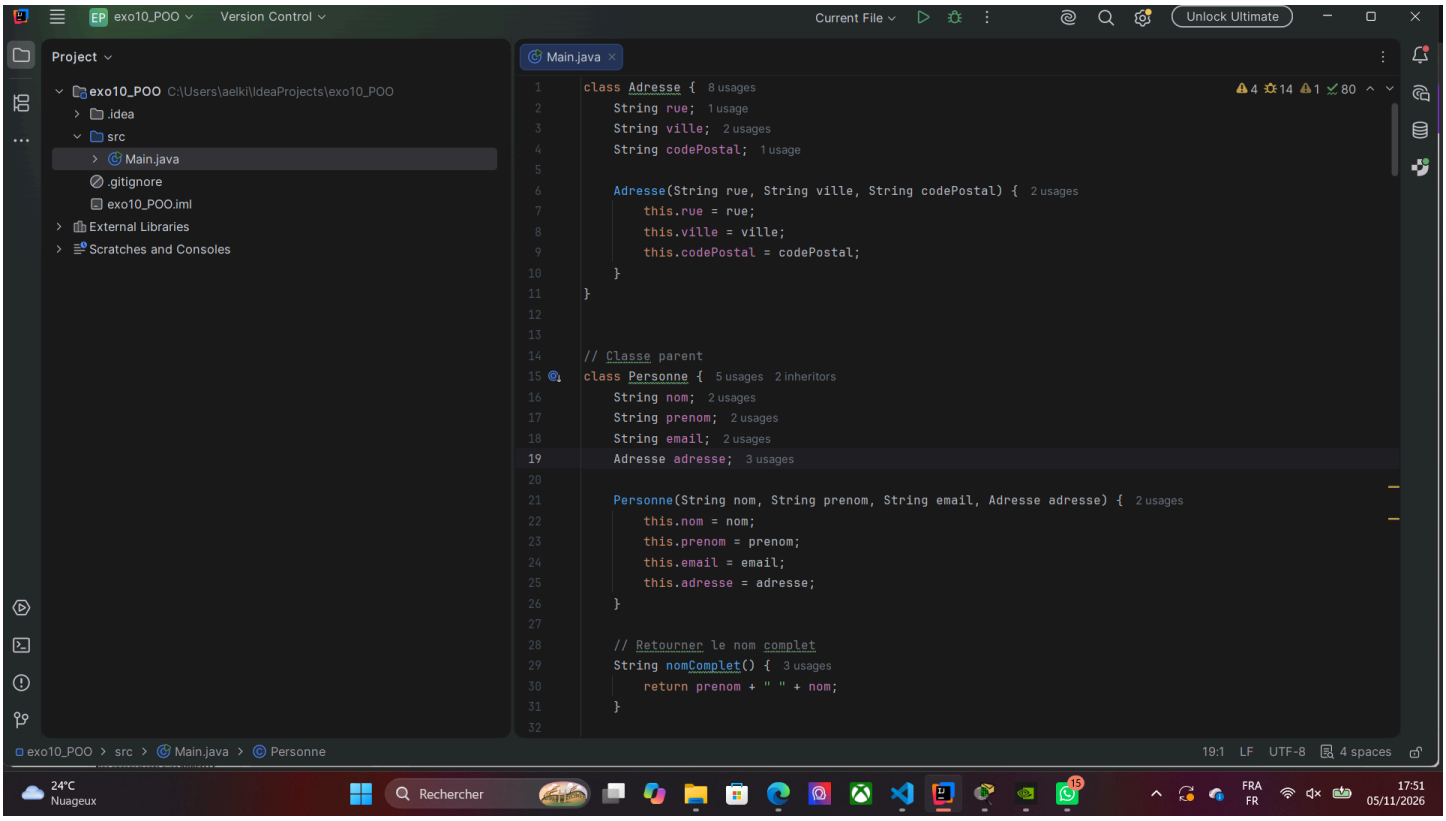
Salma pense qu'elle est inscrite à JAVA.

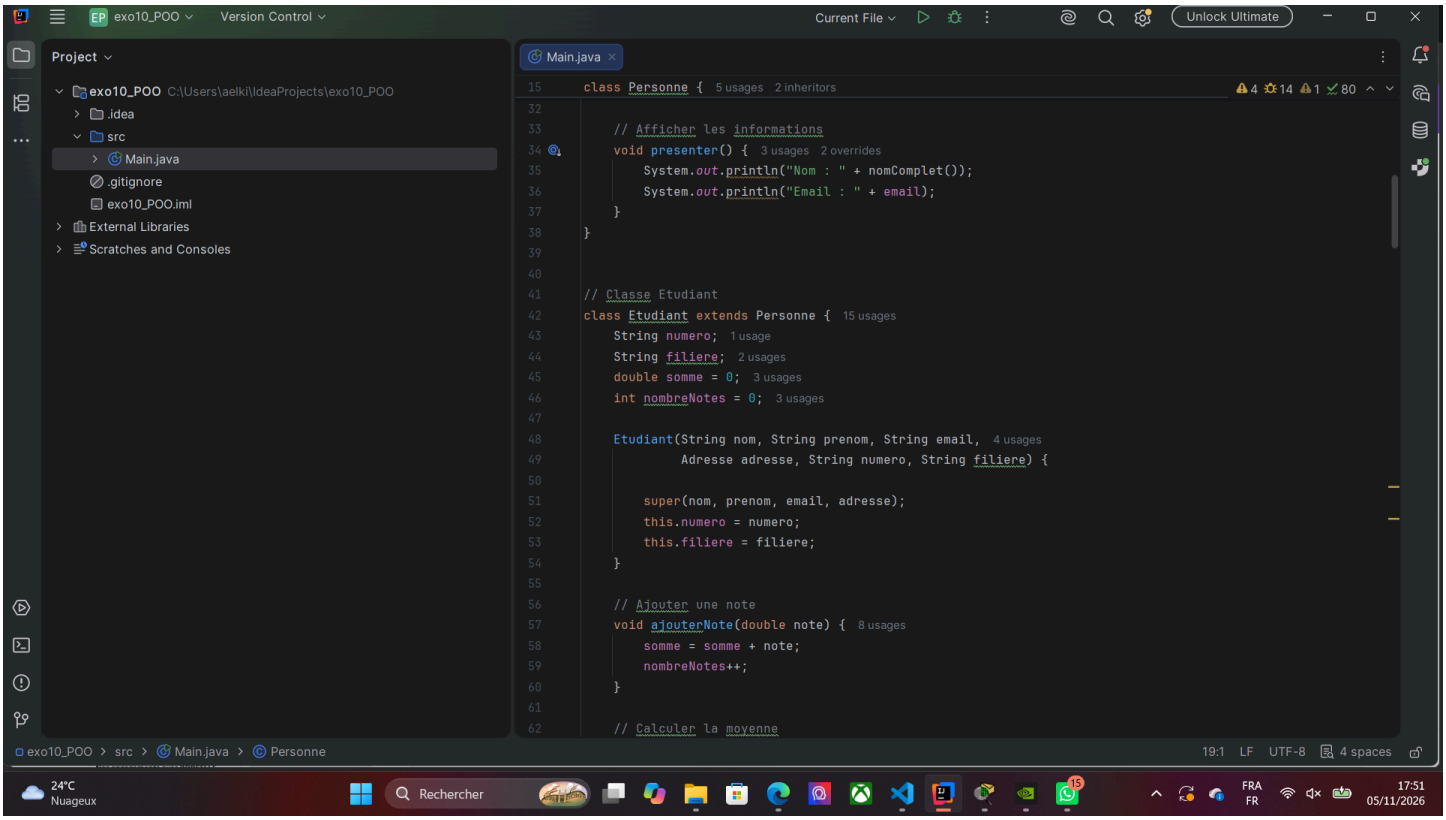
JAVA ne reconnaît pas Salma comme étudiante inscrite.

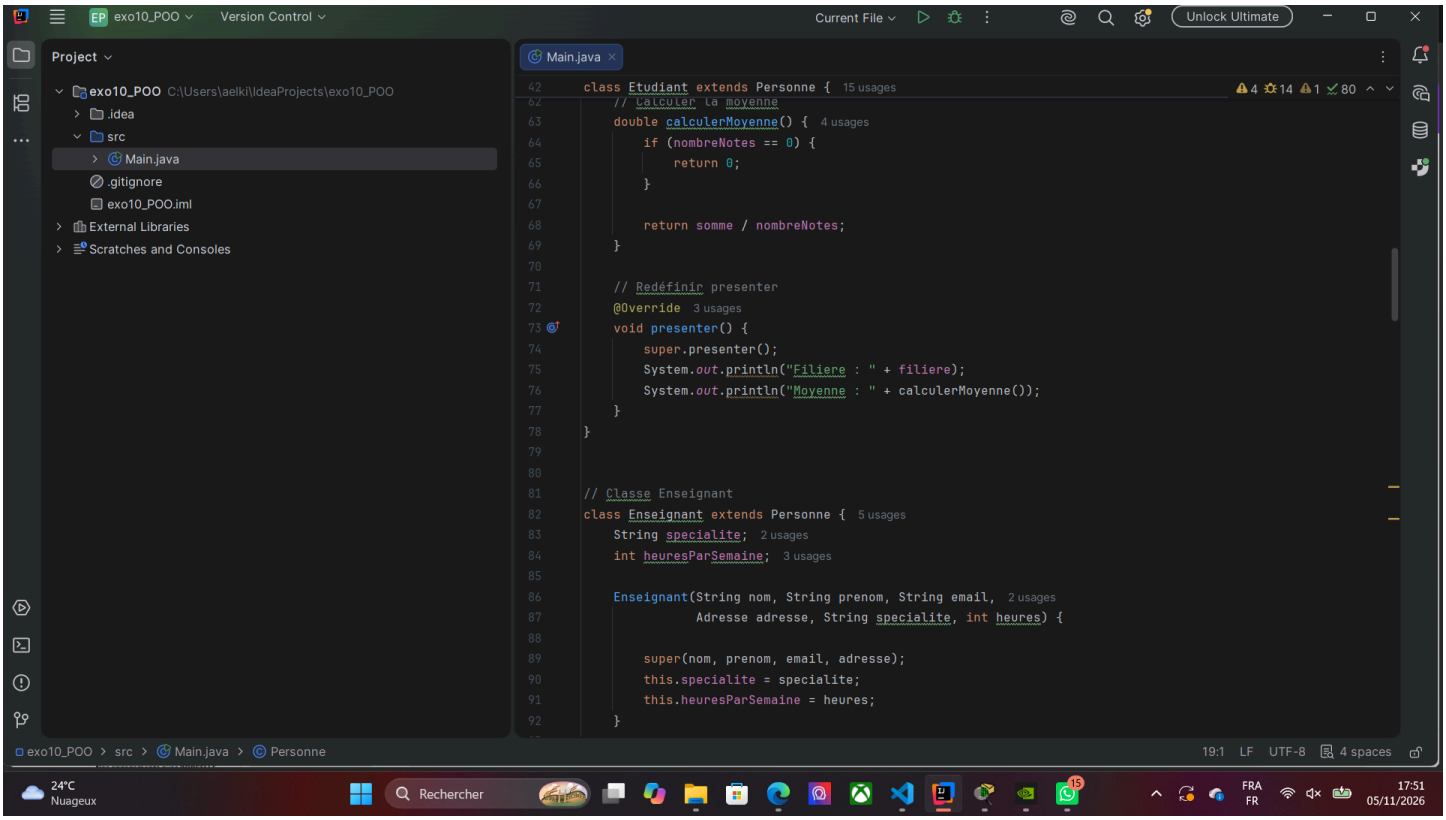
Il est donc préférable d'utiliser une seule méthode inscrire() pour mettre à jour les deux côtés en même temps.

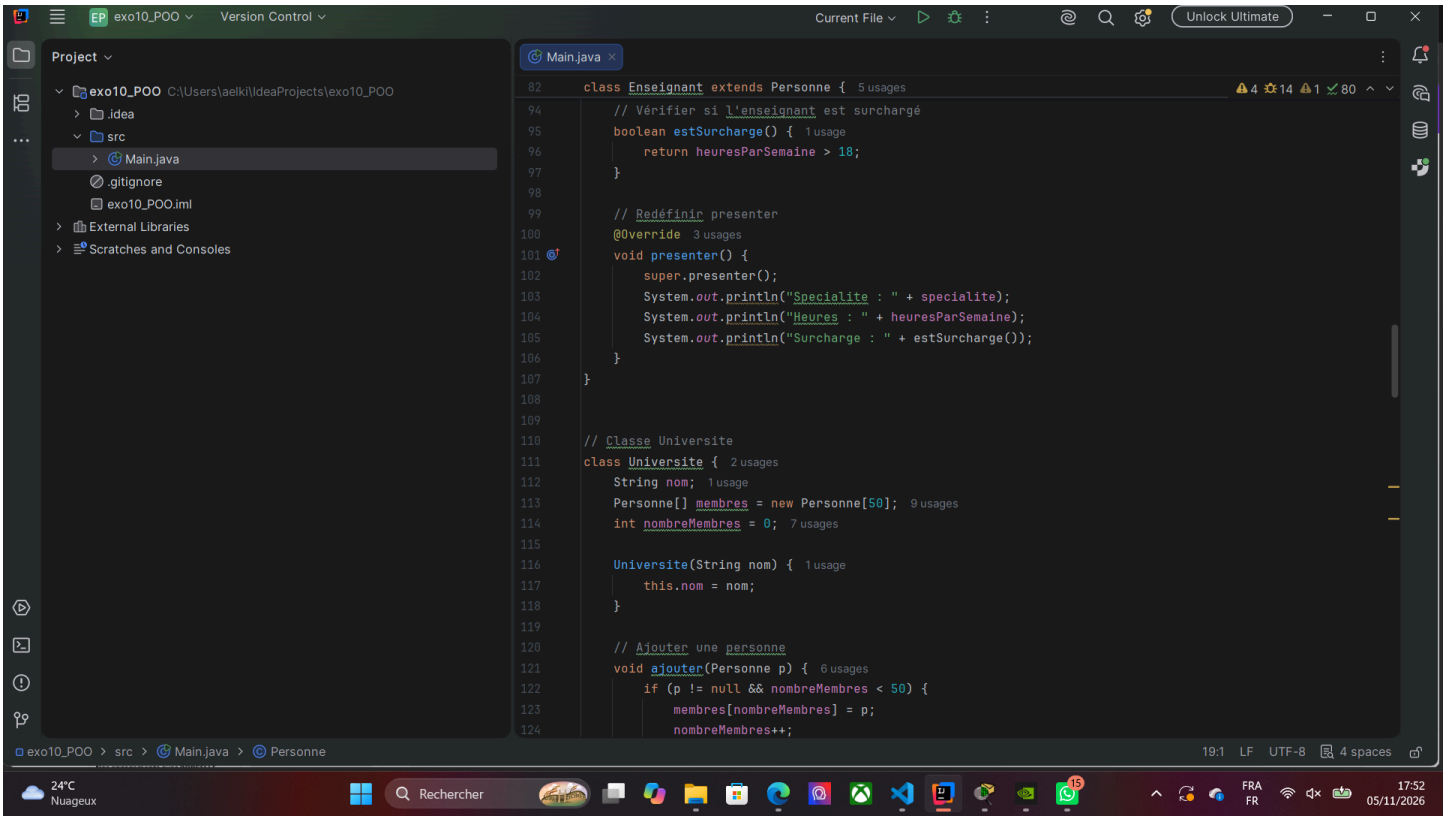
Cela permet de garantir la cohérence des données et d'éviter les erreurs.

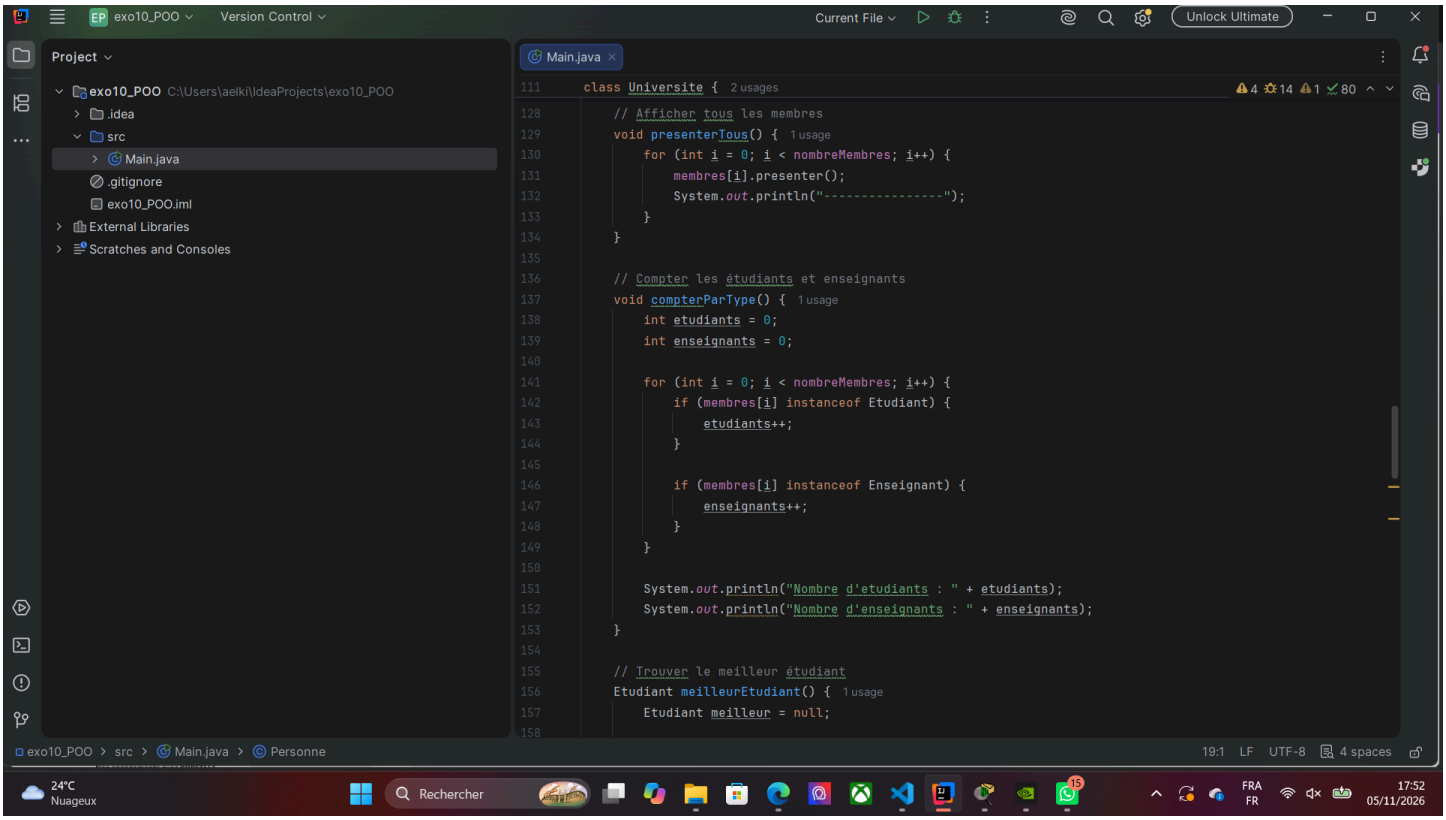
EXO 10 :

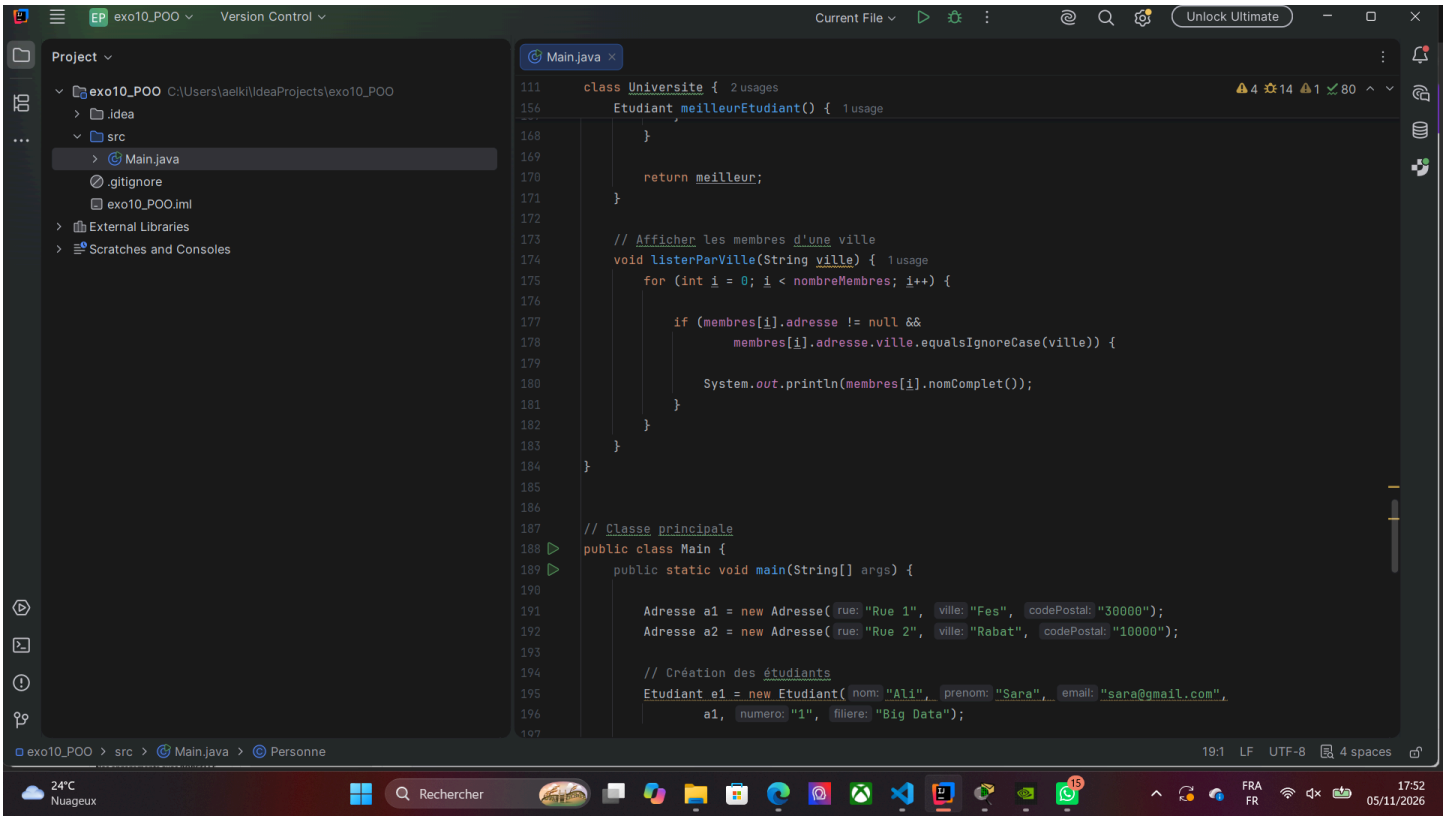


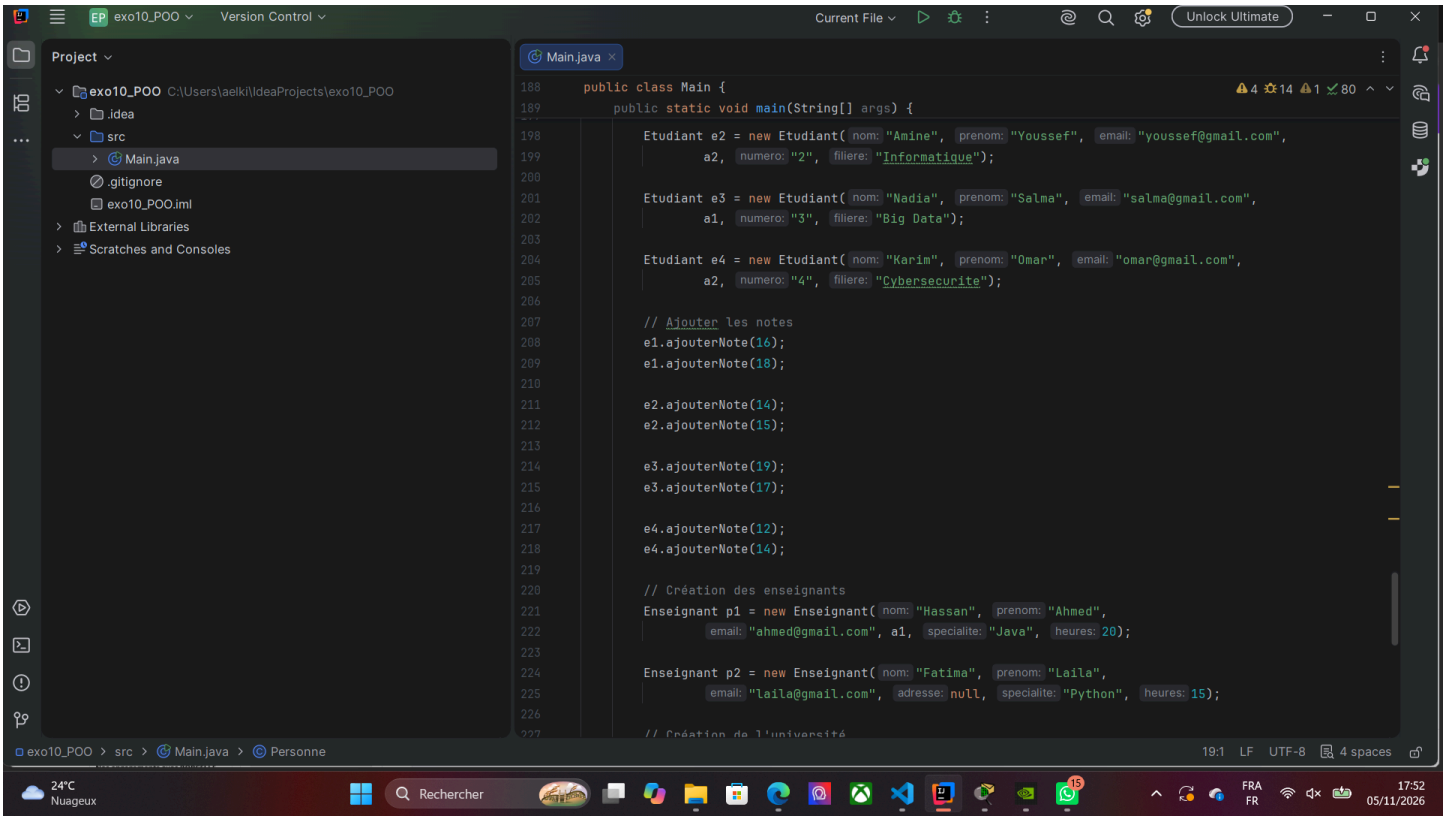

exo10_POO
Version Control
Current File
Unlock Ultimate
Project
exo10_POO C:\Users\aelki\IdeaProjects\exo10_POO
.idea
src
Main.java
.gitignore
exo10_POO.iml
External Libraries
Scratches and Consoles
Main.java
class Personne { 5 usages 2 inheritors
// Afficher les informations
void presenter() { 3 usages 2 overrides
System.out.println("Nom : " + nomComplet());
System.out.println("Email : " + email);
}
}
// Classe Etudiant
class Etudiant extends Personne { 15 usages
String numero; 1 usage
String filiere; 2 usages
double somme = 0; 3 usages
int nombreNotes = 0; 3 usages
Etudiant(String nom, String prenom, String email, 4 usages
Adresse adresse, String numero, String filiere) {
super(nom, prenom, email, adresse);
this.numero = numero;
this.filiere = filiere;
}
// Ajouter une note
void ajouterNote(double note) { 8 usages
somme = somme + note;
nombreNotes++;
}
// Calculer la moyenne
exo10_POO > src > Main.java > Personne
19:1 LF UTF-8 4 spaces
24°C
Nuageux
Rechercher
FRA
FR
17:51
05/11/2026

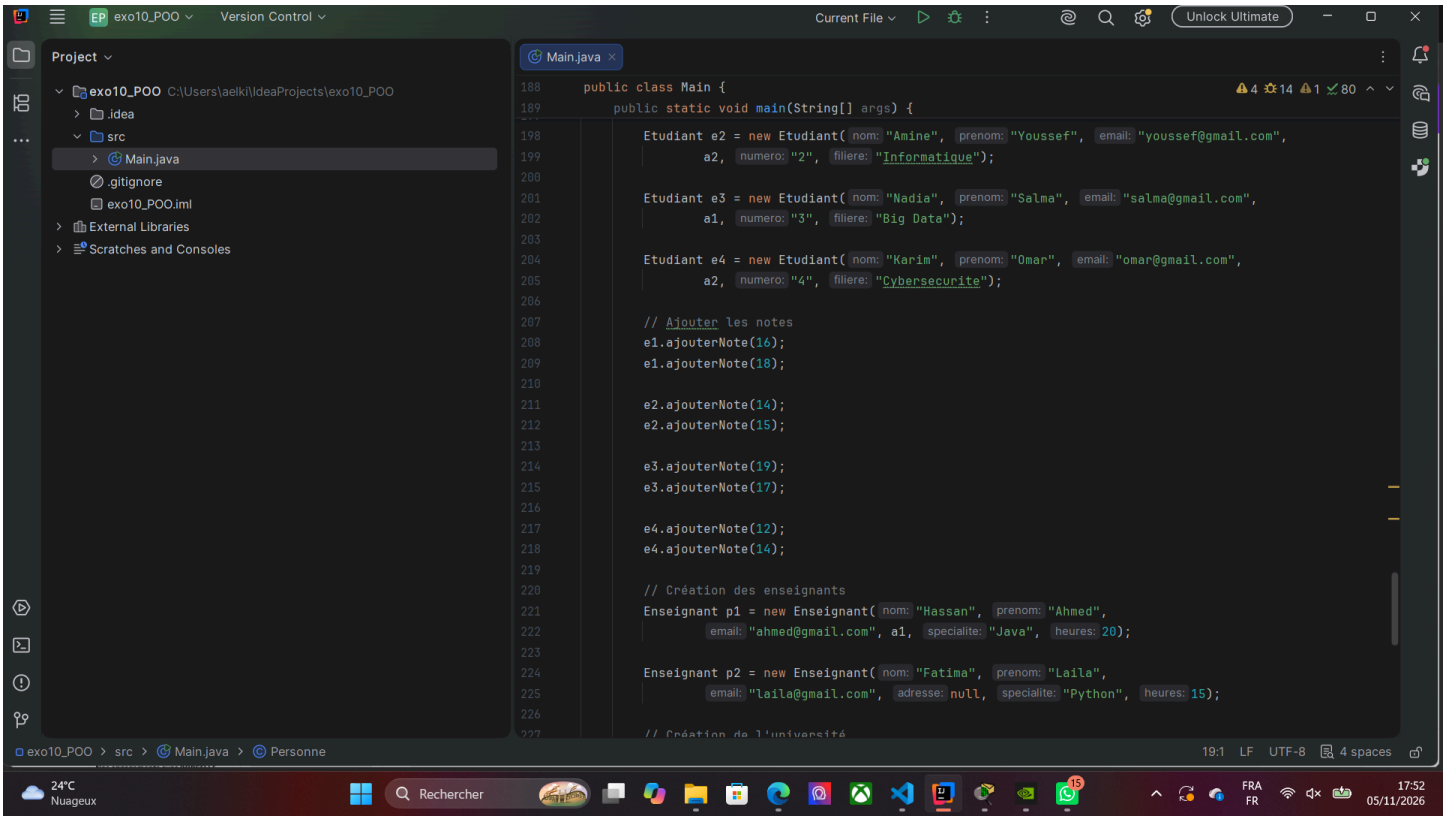

exo10_POO
Version Control
Current File
Unlock Ultimate
Project
exo10_POO C:\Users\aelki\IdeaProjects\exo10_POO
.idea
src
Main.java
.gitignore
exo10_POO.iml
External Libraries
Scratches and Consoles
Main.java
class Etudiant extends Personne { 15 usages
// Calculer la moyenne
double calculerMoyenne() { 4 usages
if (nombreNotes == 0) {
return 0;
}
return somme / nombreNotes;
}
// Redéfinir presenter
@Override 3 usages
void presenter() {
super.presenter();
System.out.println("Filiere : " + filiere);
System.out.println("Moyenne : " + calculerMoyenne());
}
}
// Classe Enseignant
class Enseignant extends Personne { 5 usages
String specialite; 2 usages
int heuresParSemaine; 3 usages
Enseignant(String nom, String prenom, String email, 2 usages
Adresse adresse, String specialite, int heures) {
super(nom, prenom, email, adresse);
this.specialite = specialite;
this.heuresParSemaine = heures;
}
exo10_POO › src › Main.java › Personne
19:1 LF UTF-8 4 spaces
24°C
Nuageux
Rechercher
FRA
FR
17:51
05/11/2026


exo10_POO
Version Control
Current File
Unlock Ultimate
Project
exo10_POO C:\Users\aelki\IdeaProjects\exo10_POO
.idea
src
Main.java
.gitignore
exo10_POO.iml
External Libraries
Scratches and Consoles
Main.java
class Enseignant extends Personne { 5 usages
// Vérifier si l'enseignant est surchargé
boolean estSurcharge() { 1 usage
return heuresParSemaine > 18;
}
// Redéfinir presenter
@Override 3 usages
void presenter() {
super.presenter();
System.out.println("Specialite : " + specialite);
System.out.println("Heures : " + heuresParSemaine);
System.out.println("Surcharge : " + estSurcharge());
}
}
// Classe Universite
class Universite { 2 usages
String nom; 1 usage
Personne[] membres = new Personne[50]; 9 usages
int nombreMembres = 0; 7 usages
Universite(String nom) { 1 usage
this.nom = nom;
}
// Ajouter une personne
void ajouter(Personne p) { 6 usages
if (p != null && nombreMembres < 50) {
membres[nombreMembres] = p;
nombreMembres++;
exo10_POO › src › Main.java › Personne
19:1 LF UTF-8 4 spaces
24°C
Nuageux
Rechercher
FRA
FR
17:52
05/11/2026


exo10_POO
Version Control
Current File
Unlock Ultimate
Project
exo10_POO C:\Users\aelki\IdeaProjects\exo10_POO
.idea
src
Main.java
.gitignore
exo10_POO.iml
External Libraries
Scratches and Consoles
Main.java
class Universite { 2 usages
// Afficher tous les membres
void presenterTous() { 1 usage
for (int i = 0; i < nombreMembres; i++) {
membres[i].presenter();
System.out.println("----------");
}
}
// Compter les étudiants et enseignants
void compterParType() { 1 usage
int etudiants = 0;
int enseignants = 0;
for (int i = 0; i < nombreMembres; i++) {
if (membres[i] instanceof Etudiant) {
etudiants++;
}
if (membres[i] instanceof Enseignant) {
enseignants++;
}
}
System.out.println("Nombre d'etudiants : " + etudiants);
System.out.println("Nombre d'enseignants : " + enseignants);
}
// Trouver le meilleur étudiant
Etudiant meilleurEtudiant() { 1 usage
Etudiant meilleur = null;
exo10_POO > src > Main.java > Personne
19:1 LF UTF-8 4 spaces
24°C
Nuageux
Rechercher
FRA
FR
17:52
05/11/2026


exo10_POO
Version Control
Current File
Unlock Ultimate
Project
exo10_POO C:\Users\aelki\IdeaProjects\exo10_POO
.idea
src
Main.java
.gitignore
exo10_POO.iml
External Libraries
Scratches and Consoles
Main.java
class Universite { 2 usages
Etudiant meilleurEtudiant() { 1 usage
}
return meilleur;
}
// Afficher les membres d'une ville
void listerParVille(String ville) { 1 usage
for (int i = 0; i < nombreMembres; i++) {
if (membres[i].adresse != null &&
membres[i].adresse.ville.equalsIgnoreCase(ville)) {
System.out.println(membres[i].nomComplet());
}
}
}
}
// Classe principale
public class Main {
public static void main(String[] args) {
Adresse a1 = new Adresse(rue: "Rue 1", ville: "Fes", codePostal: "30000");
Adresse a2 = new Adresse(rue: "Rue 2", ville: "Rabat", codePostal: "10000");
// Création des étudiants
Etudiant e1 = new Etudiant(nom: "Ali", prenom: "Sara", email: "sara@gmail.com",
a1, numero: "1", filiere: "Big Data");
exo10_POO > src > Main.java > Personne
19:1 LF UTF-8 4 spaces
24°C
Nuageux
Rechercher
FRA
FR
17:52
05/11/2026


exo10_POO
Version Control
Current File
Unlock Ultimate
Project
exo10_POO C:\Users\aelki\IdeaProjects\exo10_POO
.idea
src
Main.java
.gitignore
exo10_POO.iml
External Libraries
Scratches and Consoles
Main.java
public class Main {
public static void main(String[] args) {
Etudiant e2 = new Etudiant(nom: "Amine", prenom: "Youssef", email: "youssef@gmail.com",
a2, numero: "2", filiere: "Informatique");
Etudiant e3 = new Etudiant(nom: "Nadia", prenom: "Salma", email: "salma@gmail.com",
a1, numero: "3", filiere: "Big Data");
Etudiant e4 = new Etudiant(nom: "Karim", prenom: "Omar", email: "omar@gmail.com",
a2, numero: "4", filiere: "Cybersecurite");
// Ajouter les notes
e1.ajouterNote(16);
e1.ajouterNote(18);
e2.ajouterNote(14);
e2.ajouterNote(15);
e3.ajouterNote(19);
e3.ajouterNote(17);
e4.ajouterNote(12);
e4.ajouterNote(14);
// Création des enseignants
Enseignant p1 = new Enseignant(nom: "Hassan", prenom: "Ahmed",
email: "ahmed@gmail.com", a1, specialite: "Java", heures: 20);
Enseignant p2 = new Enseignant(nom: "Fatima", prenom: "Laila",
email: "laila@gmail.com", adresse: null, specialite: "Python", heures: 15);
exo10_POO > src > Main.java > Personne
19:1 LF UTF-8 4 spaces
24°C
Nuageux
Rechercher
FRA
FR
17:52
05/11/2026


exo10_POO
Version Control
Current File
Unlock Ultimate
Project
exo10_POO C:\Users\aelki\IdeaProjects\exo10_POO
.idea
src
Main.java
.gitignore
exo10_POO.iml
External Libraries
Scratches and Consoles
Main.java
public class Main {
public static void main(String[] args) {
Etudiant e2 = new Etudiant(nom: "Amine", prenom: "Youssef", email: "youssef@gmail.com",
a2, numero: "2", filiere: "Informatique");
Etudiant e3 = new Etudiant(nom: "Nadia", prenom: "Salma", email: "salma@gmail.com",
a1, numero: "3", filiere: "Big Data");
Etudiant e4 = new Etudiant(nom: "Karim", prenom: "Omar", email: "omar@gmail.com",
a2, numero: "4", filiere: "Cybersecurite");
// Ajouter les notes
e1.ajouterNote(16);
e1.ajouterNote(18);
e2.ajouterNote(14);
e2.ajouterNote(15);
e3.ajouterNote(19);
e3.ajouterNote(17);
e4.ajouterNote(12);
e4.ajouterNote(14);
// Création des enseignants
Enseignant p1 = new Enseignant(nom: "Hassan", prenom: "Ahmed",
email: "ahmed@gmail.com", a1, specialite: "Java", heures: 20);
Enseignant p2 = new Enseignant(nom: "Fatima", prenom: "Laila",
email: "laila@gmail.com", adresse: null, specialite: "Python", heures: 15);
exo10_POO > src > Main.java > Personne
19:1 LF UTF-8 4 spaces
24°C
Nuageux
Rechercher
FRA
FR
17:52
05/11/2026

```java
        // Liste des membres de Fes
        System.out.println("=== MEMBRES DE FES ===");
        u.listerParVille("Fes");
    }
}
```

Questions de réflexion :

Pourquoi peut-on ranger un Etudiant dans une case de type Personne, alors que calculerMoyenne() ?

Réponse à mettre dans le compte rendu :

On peut ranger un étudiant dans un tableau de type Personne car Etudiant hérite de Personne. Cependant, la méthode calculerMoyenne() appartient uniquement à la classe Etudiant. Il faut donc utiliser un cast pour accéder à cette méthode.

Dans meilleurEtudiant(), pourquoi faut-il écrire (Etudiant) membres[i] ?

Parce que membres[i] est de type Personne. Le cast permet de convertir la référence en Etudiant pour utiliser ses méthodes spécifiques, comme calculerMoyenne().

Mini-projet : station de capteurs

Analyse et conception du système

Le système est organisé autour de cinq classes principales :

1. **Mesure : représente une valeur mesurée avec son horodatage.**
2. **Capteur : classe parent contenant les informations communes aux capteurs.**
3. **CapteurTemperature : gère les mesures de température.**
4. **CapteurHumidite : gère les mesures d'humidité.**
5. **CapteurQualiteAir : gère les mesures de qualité de l'air.**
6. **Station : regroupe les capteurs et permet de gérer leurs données.**

Les trois classes spécialisées héritent de la classe Capteur.

Tableau 1-classe

Classe	Attributs principaux	Rôle
Mesure	horodatage, valeur	Enregistrer une mesure et sa date
Capteur	identifiant, emplacement, unite, mesures	Gérer les mesures et calculer les statistiques
CapteurTemperature	seuil	Détecter une température trop élevée
CapteurHumidite	Aucun attribut supplémentaire	Détecter une humidité anormale
CapteurQualiteAir	Aucun attribut supplémentaire	Évaluer la qualité de l'air
Station	nom, ville, capteurs	Gérer les capteurs et produire le rappo

Tableau 2 — Conditions de déclenchement des alertes

Type de capteur	Condition	Résultat
Température	Dernière mesure supérieure au seuil fixé (35 °C dans le test)	Alerte si la condition est vraie
Humidité	Dernière mesure inférieure à 20 % ou supérieure à 90 %	Alerte si la condition est vraie
Qualité de l’air (AQI)	Dernière mesure supérieure à 150	Alerte si la condition est vraie

Main.java

```java
import java.util.Random;
import java.util.Scanner;

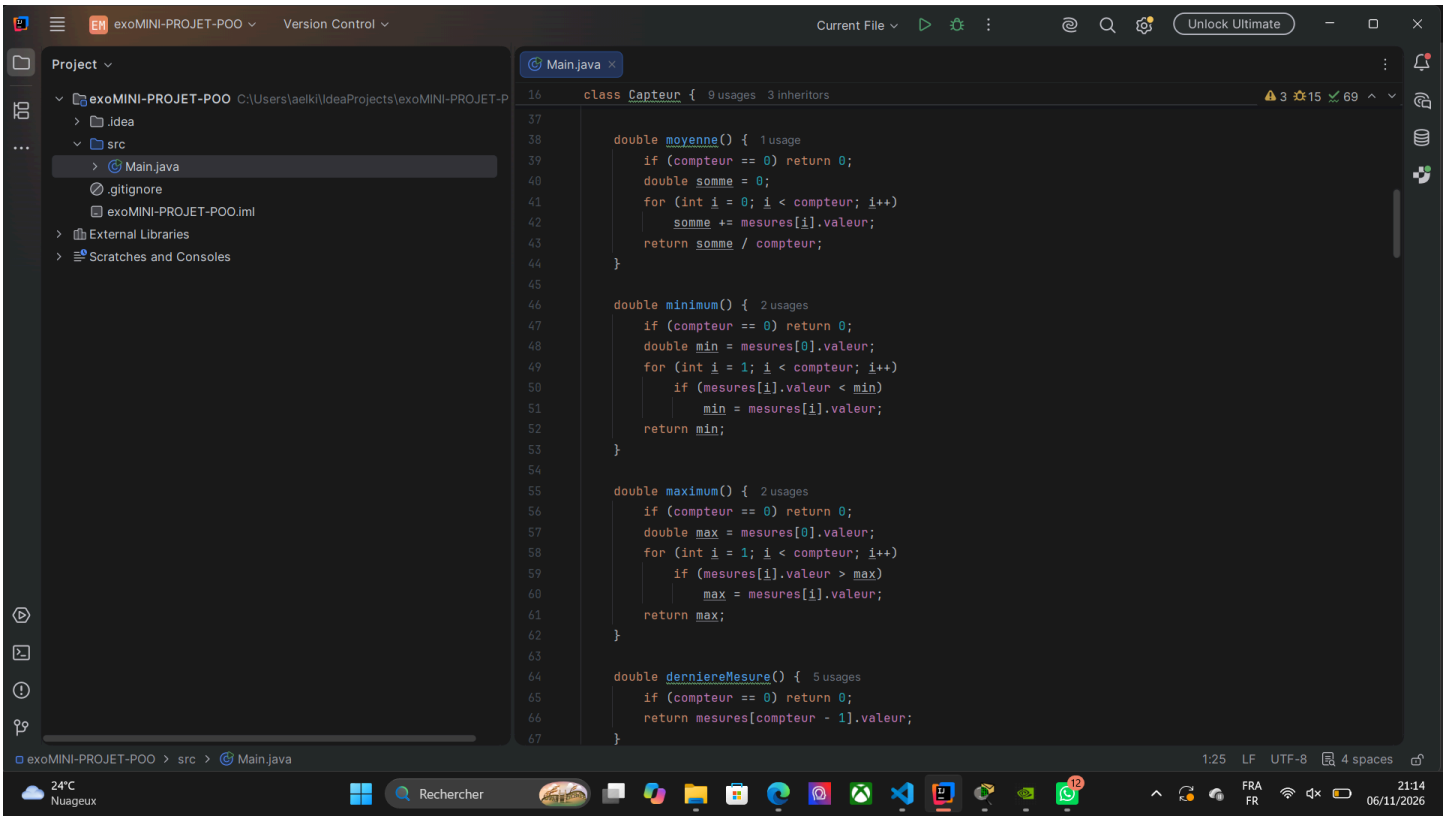
// Classe Mesure
class Mesure {  3 usages
    String horodatage;  1 usage
    double valeur;  9 usages

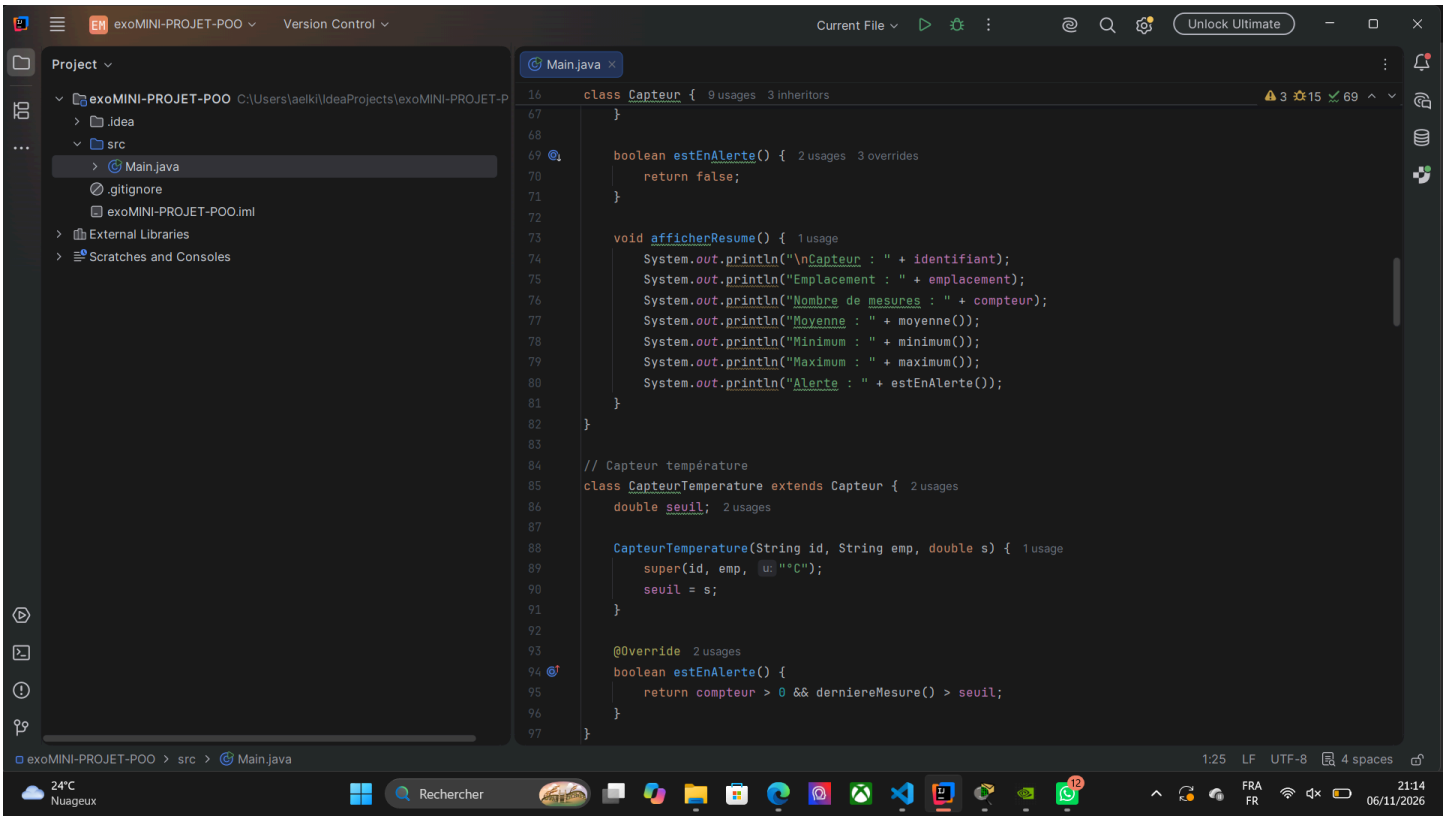
    Mesure(String h, double v) {  1 usage
        horodatage = h;
        valeur = v;
    }
}

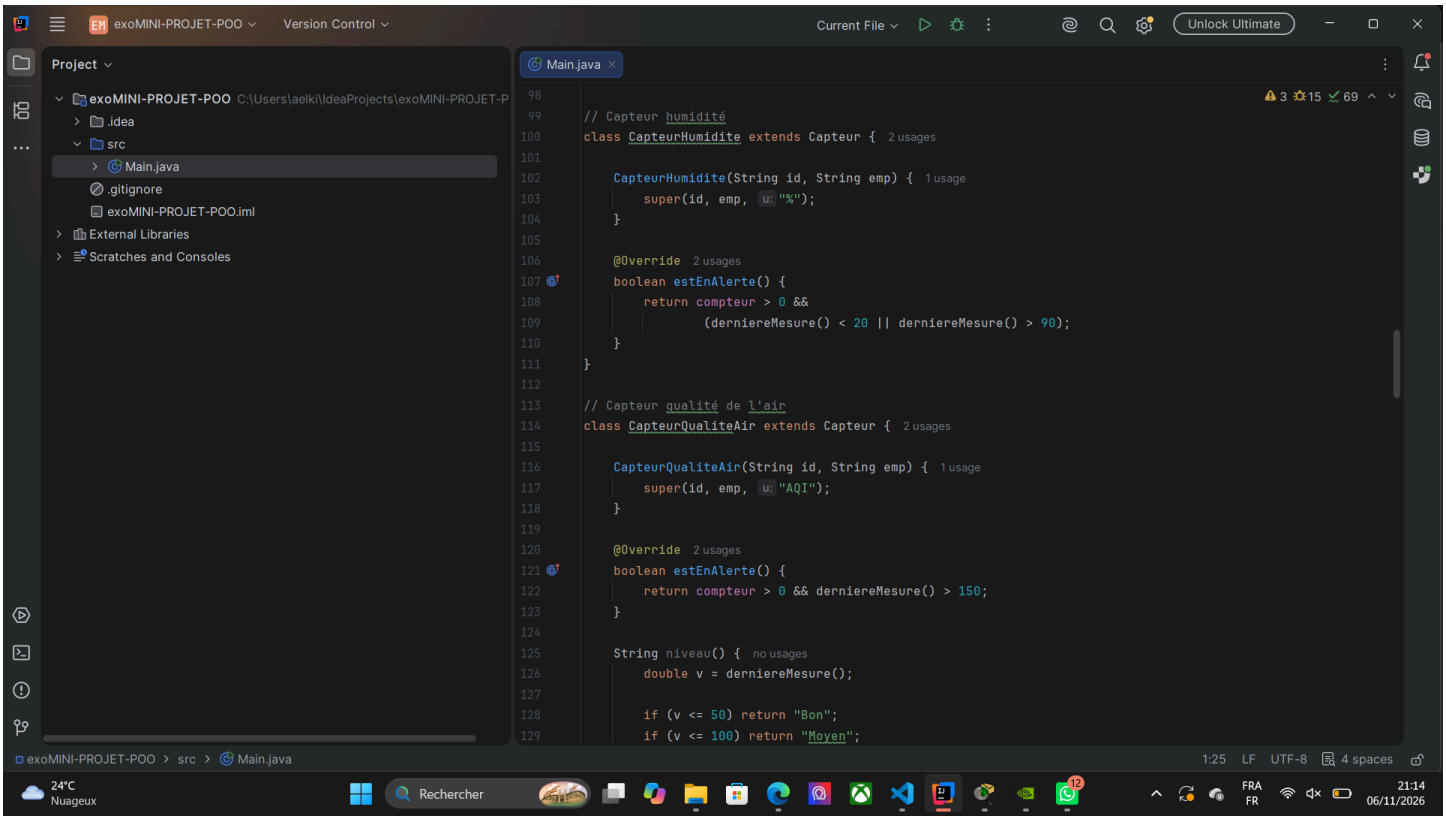
// Classe parent Capteur
class Capteur {  9 usages  3 inheritors
    String identifiant;  4 usages
    String emplacement;  2 usages
    String unite;  1 usage
    Mesure[] mesures = new Mesure[100];  9 usages
    int compteur = 0;  17 usages

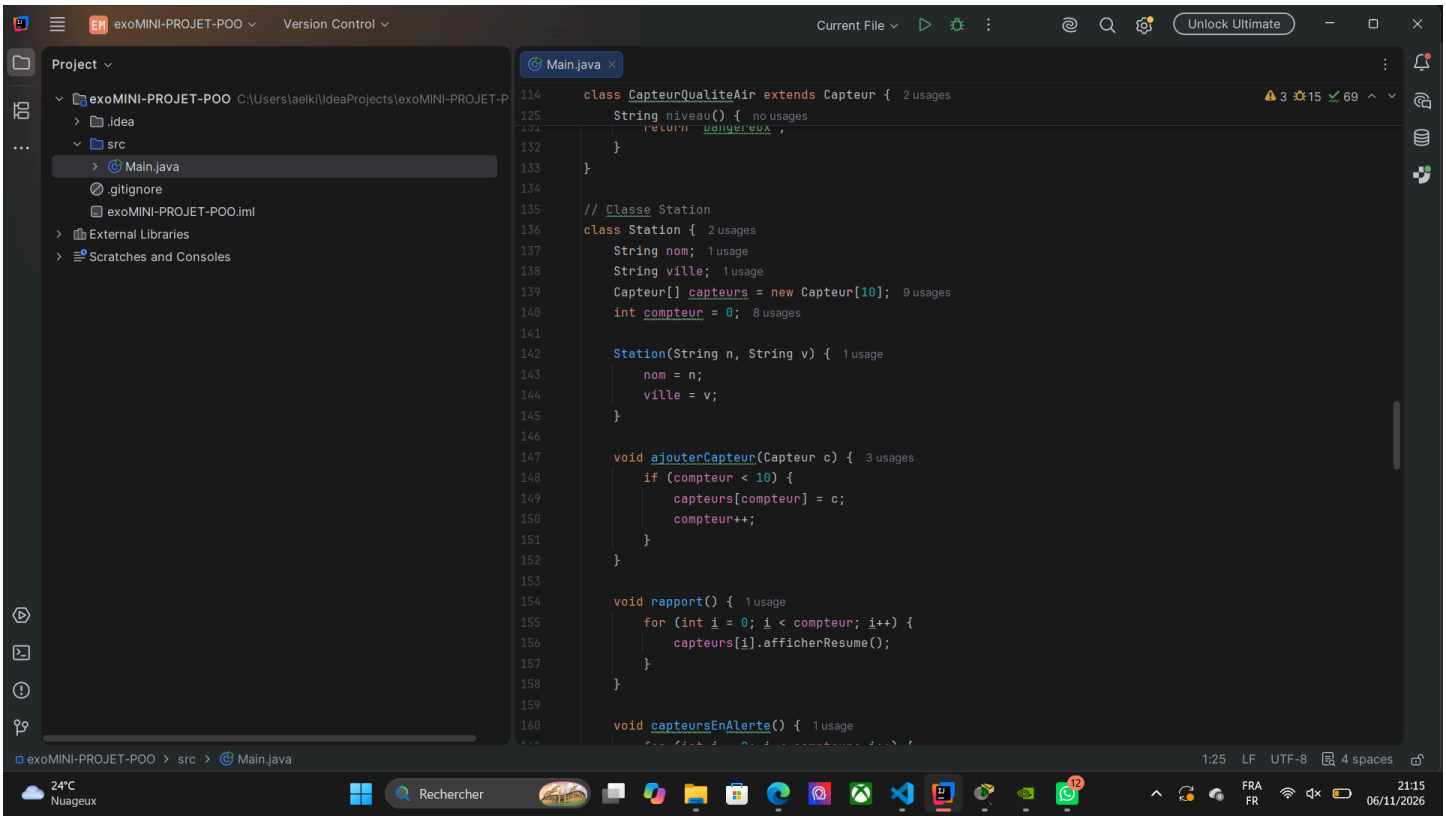
    Capteur(String id, String emp, String u) {  3 usages
        identifiant = id;
        emplacement = emp;
        unite = u;
    }

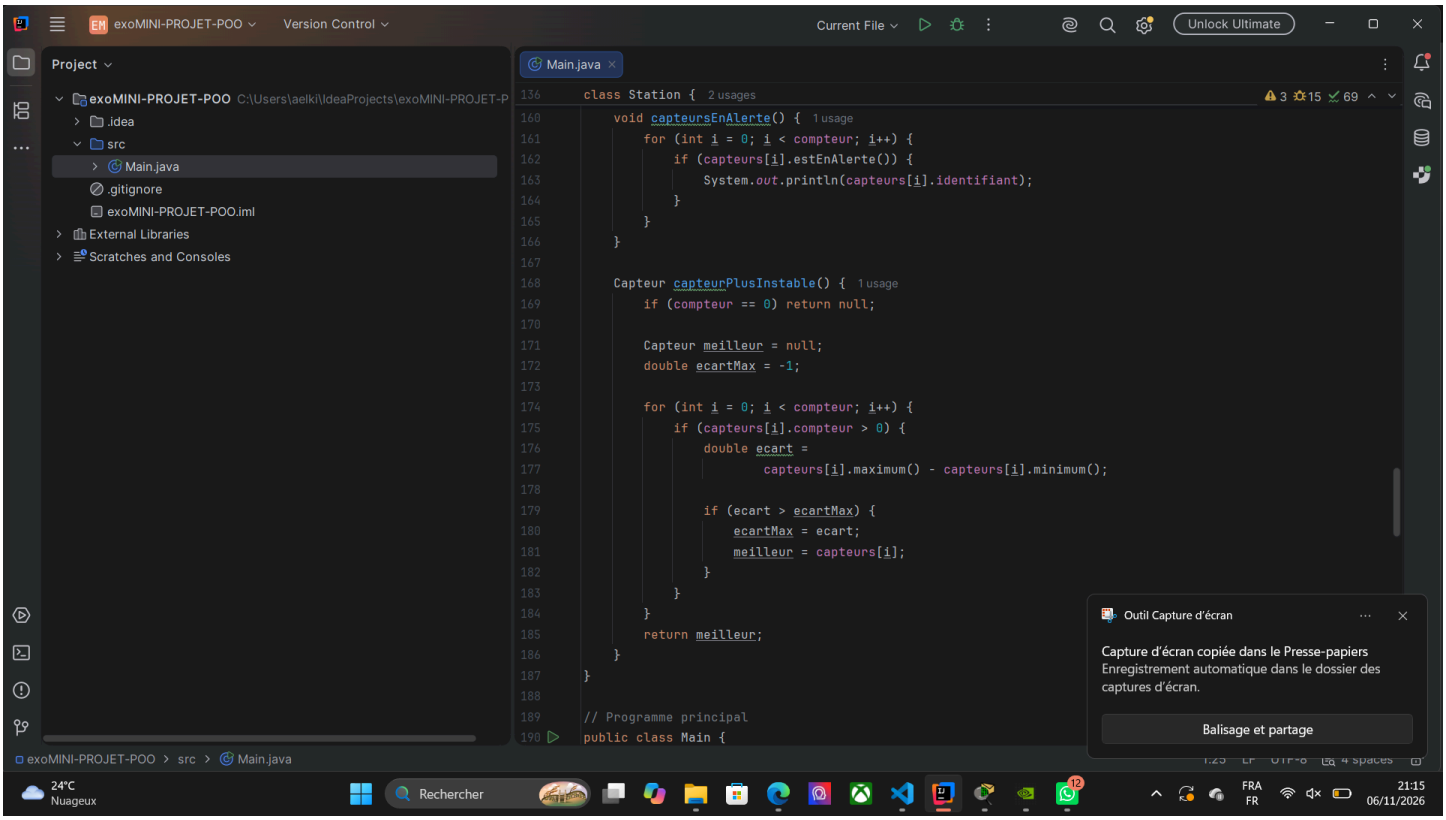
    void ajouterMesure(String date, double valeur) {  4 usages
        if (compteur < 100) {
            mesures[compteur] = new Mesure(date, valeur);
            compteur++;
```

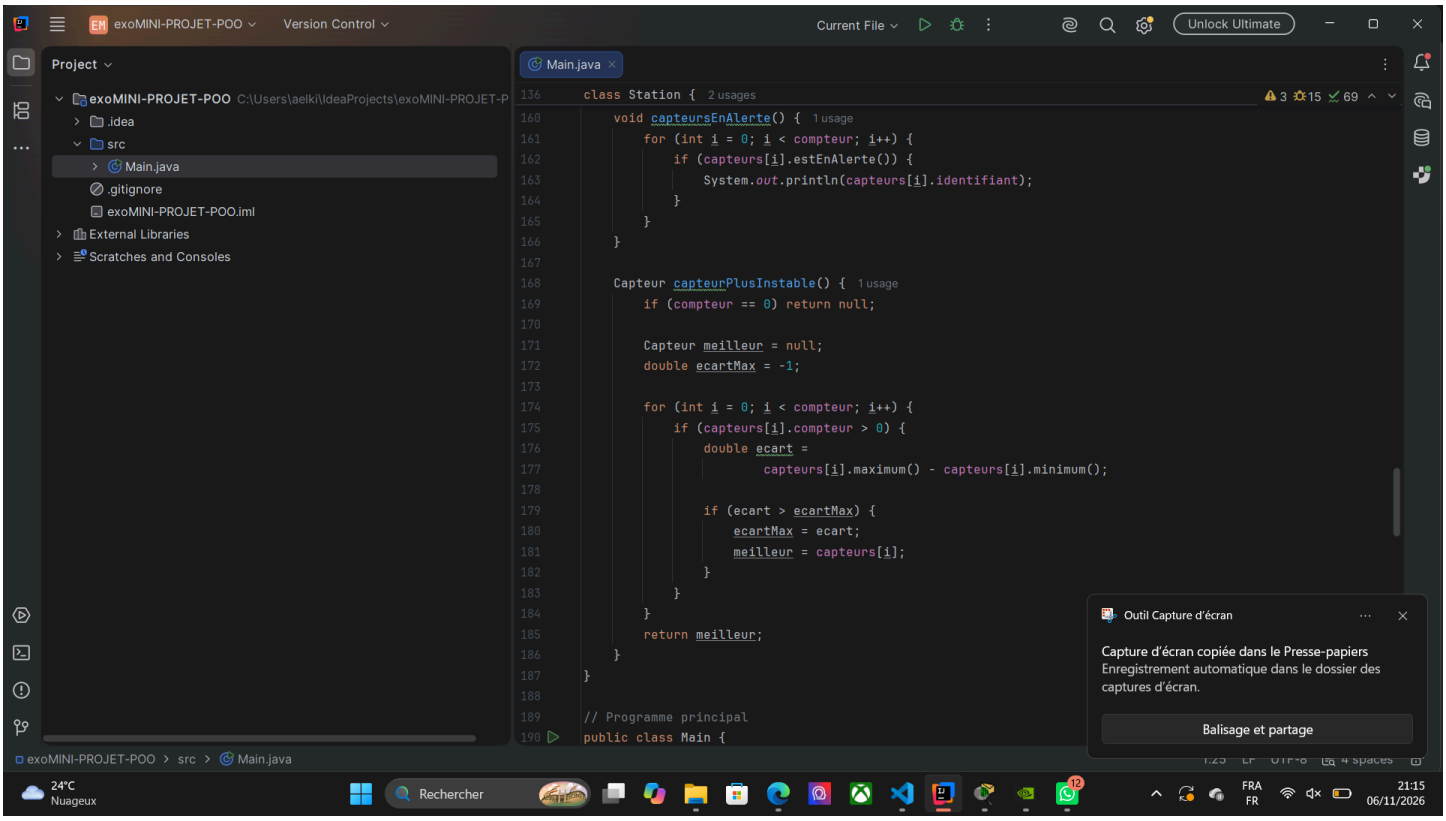

exoMINI-PROJET-POO
Version Control
Current File
Unlock Ultimate
Project
exoMINI-PROJET-POO C:\Users\aelki\IdeaProjects\exoMINI-PROJET-P
.idea
src
Main.java
.gitignore
exoMINI-PROJET-POO.iml
External Libraries
Scratches and Consoles
Main.java
class Capteur { 9 usages 3 inheritors
double moyenne() { 1 usage
if (compteur == 0) return 0;
double somme = 0;
for (int i = 0; i < compteur; i++)
somme += mesures[i].valeur;
return somme / compteur;
}
double minimum() { 2 usages
if (compteur == 0) return 0;
double min = mesures[0].valeur;
for (int i = 1; i < compteur; i++)
if (mesures[i].valeur < min)
min = mesures[i].valeur;
return min;
}
double maximum() { 2 usages
if (compteur == 0) return 0;
double max = mesures[0].valeur;
for (int i = 1; i < compteur; i++)
if (mesures[i].valeur > max)
max = mesures[i].valeur;
return max;
}
double derniereMesure() { 5 usages
if (compteur == 0) return 0;
return mesures[compteur - 1].valeur;
}
exoMINI-PROJET-POO > src > Main.java
1:25 LF UTF-8 4 spaces
24°C
Nuageux
Rechercher
FRA
FR
21:14
06/11/2026

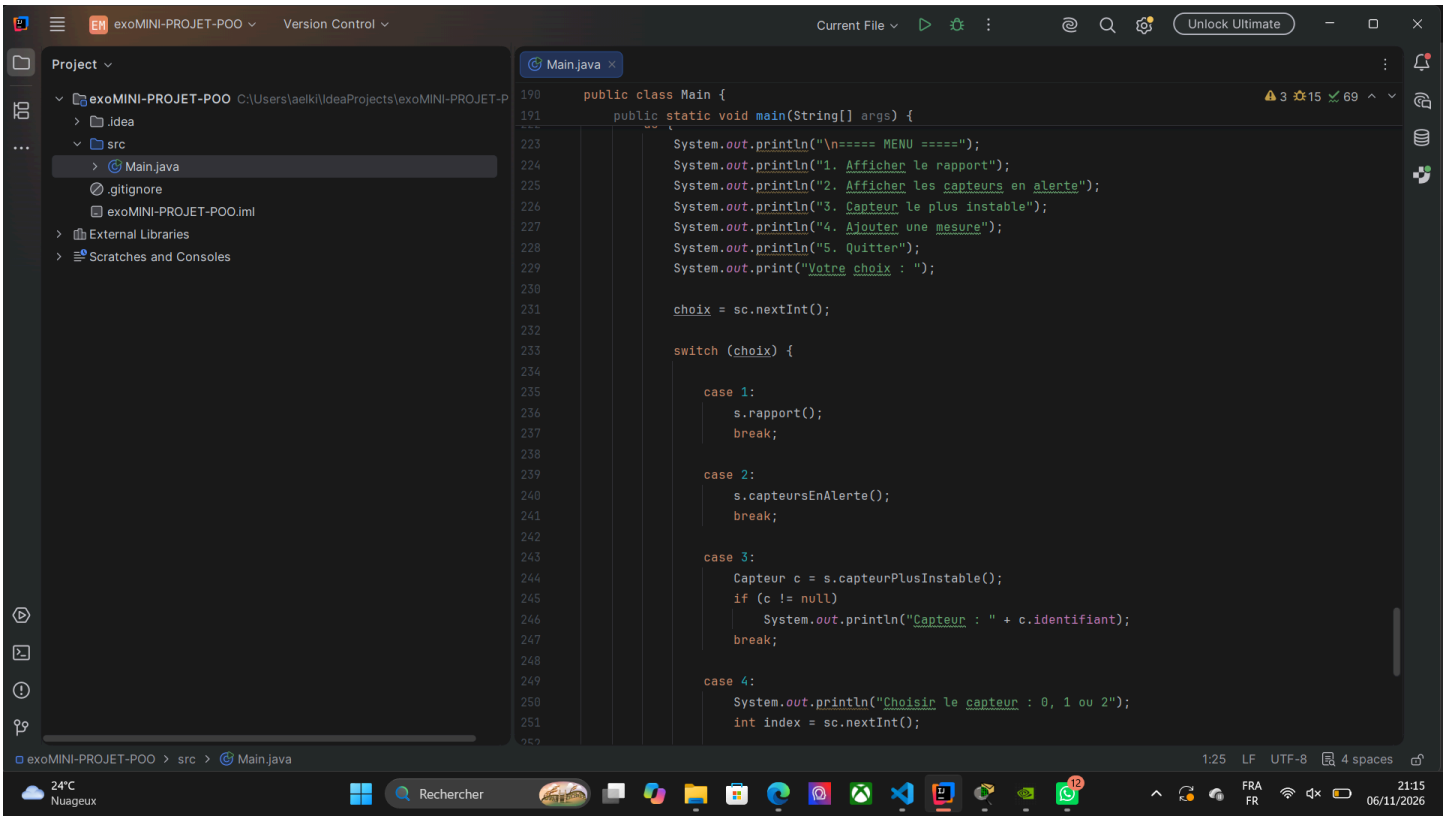

exoMINI-PROJET-POO
Version Control
Current File
Unlock Ultimate
Project
exoMINI-PROJET-POO C:\Users\aelki\IdeaProjects\exoMINI-PROJET-P
.idea
src
Main.java
.gitignore
exoMINI-PROJET-POO.iml
External Libraries
Scratches and Consoles
Main.java
class Capteur { 9 usages 3 inheritors
}
boolean estEnAlerte() { 2 usages 3 overrides
return false;
}
void afficherResume() { 1 usage
System.out.println("\nCapteur : " + identifiant);
System.out.println("Emplacement : " + emplacement);
System.out.println("Nombre de mesures : " + compteur);
System.out.println("Moyenne : " + moyenne());
System.out.println("Minimum : " + minimum());
System.out.println("Maximum : " + maximum());
System.out.println("Alerte : " + estEnAlerte());
}
}
// Capteur température
class CapteurTemperature extends Capteur { 2 usages
double seuil; 2 usages
CapteurTemperature(String id, String emp, double s) { 1 usage
super(id, emp, u: "°C");
seuil = s;
}
@Override 2 usages
boolean estEnAlerte() {
return compteur > 0 && dernieremesure() > seuil;
}
}
exoMINI-PROJET-POO > src > Main.java
1:25 LF UTF-8 4 spaces
24°C
Nuageux
Rechercher
FRA
FR
21:14
06/11/2026

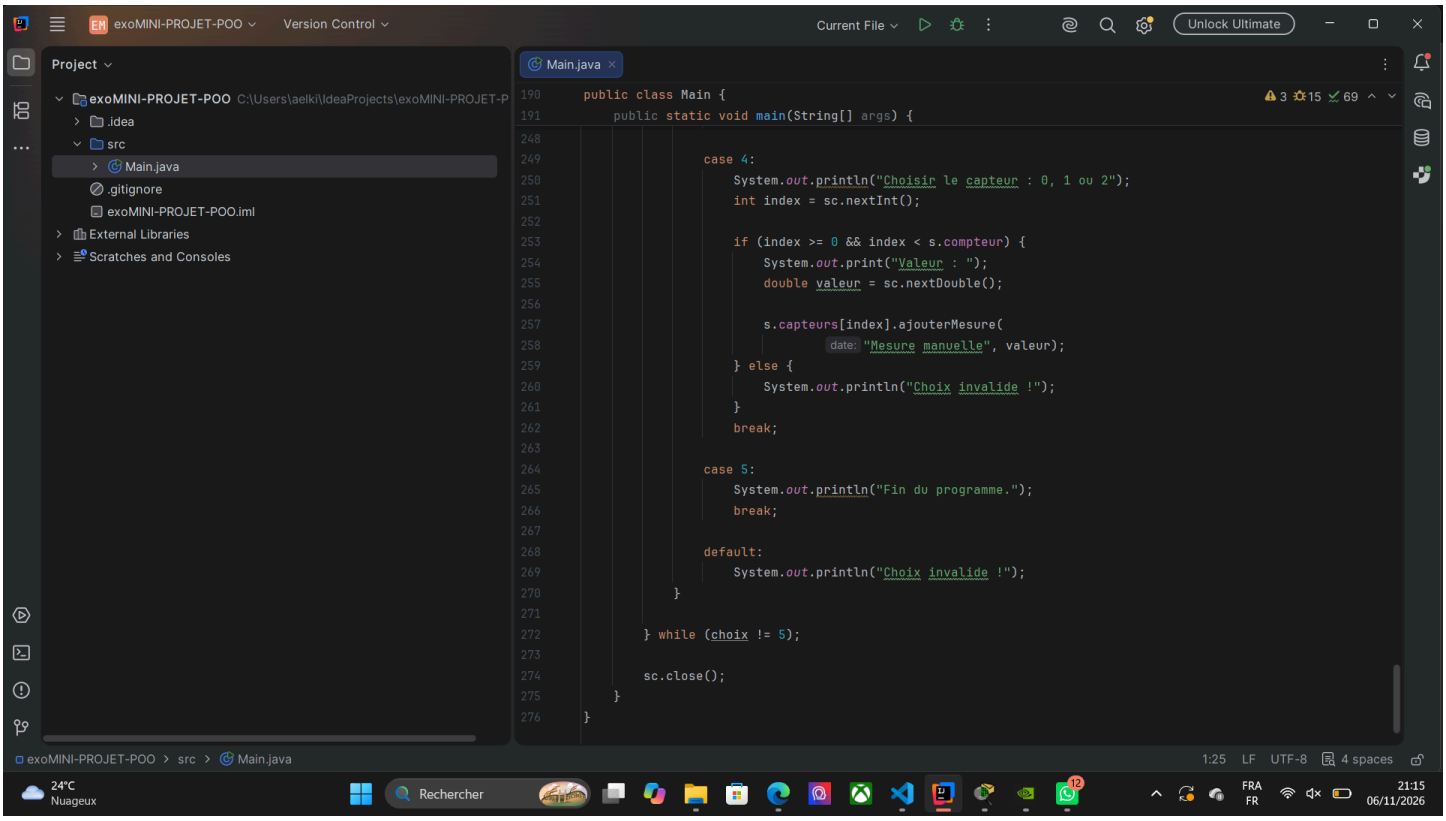

exoMINI-PROJET-POO
Version Control
Current File
Unlock Ultimate
Project
exoMINI-PROJET-POO C:\Users\aelki\IdeaProjects\exoMINI-PROJET-P
.idea
src
Main.java
.gitignore
exoMINI-PROJET-POO.iml
External Libraries
Scratches and Consoles
Main.java
// Capteur humidité
class CapteurHumidite extends Capteur { 2 usages
CapteurHumidite(String id, String emp) { 1 usage
super(id, emp, u: "%");
@Override 2 usages
boolean estEnAlerte() {
return compteur > 0 &&
(dernieremesure() < 20 || dernieremesure() > 90);
// Capteur qualité de l'air
class CapteurQualiteAir extends Capteur { 2 usages
CapteurQualiteAir(String id, String emp) { 1 usage
super(id, emp, u: "AQI");
@Override 2 usages
boolean estEnAlerte() {
return compteur > 0 && dernieremesure() > 150;
String niveau() { no usages
double v = dernieremesure();
if (v <= 50) return "Bon";
if (v <= 100) return "Moyen";
exoMINI-PROJET-POO > src > Main.java
1:25 LF UTF-8 4 spaces
24°C
Nuageux
Rechercher
FRA
FR
21:14
06/11/2026


exoMINI-PROJET-POO
Version Control
Current File
Unlock Ultimate
Project
exoMINI-PROJET-POO C:\Users\aelki\IdeaProjects\exoMINI-PROJET-P
.idea
src
Main.java
.gitignore
exoMINI-PROJET-POO.iml
External Libraries
Scratches and Consoles
Main.java
class CapteurQualiteAir extends Capteur { 2 usages
String niveau() { no usages
return "Dangereux";
// Classe Station
class Station { 2 usages
String nom; 1 usage
String ville; 1 usage
Capteur[] capteurs = new Capteur[10]; 9 usages
int compteur = 0; 8 usages
Station(String n, String v) { 1 usage
nom = n;
ville = v;
void ajouterCapteur(Capteur c) { 3 usages
if (compteur < 10) {
capteurs[compteur] = c;
compteur++;
void rapport() { 1 usage
for (int i = 0; i < compteur; i++) {
capteurs[i].afficherResume();
void capteursEnAlerte() { 1 usage
exoMINI-PROJET-POO > src > Main.java
1:25 LF UTF-8 4 spaces
24°C
Nuageux
Rechercher
FRA
FR
21:15
06/11/2026


exoMINI-PROJET-POO
Version Control
Current File
Unlock Ultimate
Project
exoMINI-PROJET-POO C:\Users\aelki\IdeaProjects\exoMINI-PROJET-P
.idea
src
Main.java
.gitignore
exoMINI-PROJET-POO.iml
External Libraries
Scratches and Consoles
Main.java
class Station { 2 usages
void capteursEnAlerte() { 1 usage
for (int i = 0; i < compteur; i++) {
if (capteurs[i].estEnAlerte()) {
System.out.println(capteurs[i].identifiant);
}
}
}
Capteur capteurPlusInstable() { 1 usage
if (compteur == 0) return null;
Capteur meilleur = null;
double ecartMax = -1;
for (int i = 0; i < compteur; i++) {
if (capteurs[i].compteur > 0) {
double ecart =
capteurs[i].maximum() - capteurs[i].minimum();
if (ecart > ecartMax) {
ecartMax = ecart;
meilleur = capteurs[i];
}
}
}
return meilleur;
}
}
// Programme principal
public class Main {
Outil Capture d'écran
Capture d'écran copiée dans le Presse-papiers
Enregistrement automatique dans le dossier des captures d'écran.
Balisage et partage
exoMINI-PROJET-POO > src > Main.java
24°C
Nuageux
Rechercher
FRA
FR
21:15
06/11/2026


exoMINI-PROJET-POO
Version Control
Current File
Unlock Ultimate
Project
exoMINI-PROJET-POO C:\Users\aelki\IdeaProjects\exoMINI-PROJET-P
.idea
src
Main.java
.gitignore
exoMINI-PROJET-POO.iml
External Libraries
Scratches and Consoles
Main.java
class Station { 2 usages
void capteursEnAlerte() { 1 usage
for (int i = 0; i < compteur; i++) {
if (capteurs[i].estEnAlerte()) {
System.out.println(capteurs[i].identifiant);
}
}
}
Capteur capteurPlusInstable() { 1 usage
if (compteur == 0) return null;
Capteur meilleur = null;
double ecartMax = -1;
for (int i = 0; i < compteur; i++) {
if (capteurs[i].compteur > 0) {
double ecart =
capteurs[i].maximum() - capteurs[i].minimum();
if (ecart > ecartMax) {
ecartMax = ecart;
meilleur = capteurs[i];
}
}
}
return meilleur;
}
}
// Programme principal
public class Main {
Outil Capture d'écran
Capture d'écran copiée dans le Presse-papiers
Enregistrement automatique dans le dossier des captures d'écran.
Balisage et partage
exoMINI-PROJET-POO > src > Main.java
24°C
Nuageux
Rechercher
FRA
FR
21:15
06/11/2026


exoMINI-PROJET-POO
Version Control
Current File
Unlock Ultimate
Project
exoMINI-PROJET-POO C:\Users\aelki\IdeaProjects\exoMINI-PROJET-P
.idea
src
Main.java
.gitignore
exoMINI-PROJET-POO.iml
External Libraries
Scratches and Consoles
Main.java
public class Main {
public static void main(String[] args) {
System.out.println("\n===== MENU =====");
System.out.println("1. Afficher le rapport");
System.out.println("2. Afficher les capteurs en alerte");
System.out.println("3. Capteur le plus instable");
System.out.println("4. Ajouter une mesure");
System.out.println("5. Quitter");
System.out.print("Votre choix : ");
choix = sc.nextInt();
switch (choix) {
case 1:
s.rapport();
break;
case 2:
s.capteursEnAlerte();
break;
case 3:
Capteur c = s.capteurPlusInstable();
if (c != null)
System.out.println("Capteur : " + c.identifiant);
break;
case 4:
System.out.println("Choisir le capteur : 0, 1 ou 2");
int index = sc.nextInt();
exoMINI-PROJET-POO > src > Main.java
1:25 LF UTF-8 4 spaces
24°C
Nuageux
Rechercher
FRA
FR
21:15
06/11/2026


exoMINI-PROJET-POO
Version Control
Current File
Unlock Ultimate
Project
exoMINI-PROJET-POO C:\Users\aelki\IdeaProjects\exoMINI-PROJET-P
.idea
src
Main.java
.gitignore
exoMINI-PROJET-POO.iml
External Libraries
Scratches and Consoles
Main.java
public class Main {
public static void main(String[] args) {
case 4:
System.out.println("Choisir le capteur : 0, 1 ou 2");
int index = sc.nextInt();
if (index >= 0 && index < s.compteur) {
System.out.print("Valeur : ");
double valeur = sc.nextDouble();
s.capteurs[index].ajouterMesure(
date: "Mesure manuelle", valeur);
} else {
System.out.println("Choix invalide !");
}
break;
case 5:
System.out.println("Fin du programme.");
break;
default:
System.out.println("Choix invalide !");
}
} while (choix != 5);
sc.close();
}
}
exoMINI-PROJET-POO > src > Main.java
1:25 LF UTF-8 4 spaces
24°C
Nuageux
Rechercher
FRA
FR
21:15
06/11/2026

1- faut créer une nouvelle classe appelée CapteurVent qui hérite de la classe Capteur.

Cette classe aura ses propres attributs et sa propre méthode estEnAlerte() pour détecter une vitesse de vent anormale.

La classe Station ne nécessite pas de modification, car elle utilise déjà un tableau de type Capteur[]. Elle peut donc accepter le nouveau capteur.

2.Le tableau fixe de 100 mesures ne sera plus suffisant.

Il faudra utiliser une base de données ou un système de stockage adapté pour enregistrer un grand nombre de mesures.

Il faudra également améliorer le traitement des données pour éviter de surcharger la mémoire et permettre une analyse plus rapide.